Chapter 5

Convolutional Models for Offline Manipulator Trajectory Planning

Contents

5.1 Introduction **100**

5.2 Deep Convolutional Neural Networks **101**

5.3 CNN VAE based Learners for Robotic Trajectory Planning **104**

5.4 Implicit Sequential Learner for Robot Trajectory Planning with GAN **120**

5.5 Disentanglement learning using βVAE **135**

5.6 Summary **140**

In the world of robotics, the art of leading a robot end effector along a path over time, generally referred to as trajectory planning, has attracted attention, especially within the framework of deep neural models. However, the ensuing high number of network parameters prompts an investigation into the efficiency of densely connected networks, resulting in a reduction in streamlined performance. The utilization of convolutional feature learners enhances the discernment of hierarchical patterns and representations within trajectory data. Crucially, the study integrates unsupervised generative frameworks, particularly Variational Autoencoders and GANs, for trajectory planning, introducing a novel approach for combining planning and motor neuron control within a single entity. This strategic amalgamation harnesses the power of Generative AI, offering a comprehensive solution for orchestrating complex movements and control mechanisms in robotic systems.

5.1 Introduction

Higher-order living organisms can learn and synthesise new motions or tasks from the complex patterns of sensory information. Organisms learn a new skill or improve existing skills from the information gained during its evolution [302]. The skills are mapped from a higher-order neural representation in the brain to lower-order neuron impulses to the joints. Hence, the evolution of organisms' brains can produce tasks with a high level of complexity in a hierarchical manner [303, 304]. Deep Neural Networks have evolved in the past decade by being inspired by the neural systems in the organisms, which have different layers of hierarchy, learning new information and optimising the network accordingly. Convolutional Neural Network developed as a breakthrough in image classification, with the pioneering Le Net design by Yann LeCun in the early 1990s [305] and later improved in 1998 [306], introducing the notion of convolutional layers and pooling operations. This evolution allows deep neural networks to handle tasks with increasing complexity. In the domain of robotics, these advanced networks play a pivotal role in trajectory planning, optimising and refining robot trajectories to enhance precision and efficiency across diverse applications, ultimately contributing to improved performance and adaptability in robotic systems.

A robot trajectory denoted as T, is a mathematical sequence of poses or joint configurations evolving over time. T is represented as $\{t_1, t_2, \ldots, t_n\}$, where each t_i signifies the state of the robot at a specific moment. The choice of representation depends on the application and the task requirements. In a pose representation, t_i includes position (x_i, y_i, z_i) and orientation $(\phi_i, \theta_i, \psi_i)$ in 3D space. Alternatively, in a joint configuration representation, t_i comprises joint angles $(q_{1i}, q_{2i}, \ldots, q_{mi})$. Mathematically, the trajectory T is expressed as a function of time t: $T(t) = \{t_1(t), t_2(t), \ldots, t_n(t)\}$. The chosen representation aligns with the specific application needs, where pose representation is common for tasks emphasizing end-effector positions, while joint configuration representation is prevalent in contexts involving robot arm movements and control.

It is considered that generative deep architectures such as CNN-VAE and GAN, when applied to these trajectory representations, have the potential to learn meaningful latent structures that capture the underlying task-relevant semantics while

separating out variations in low-level kinematic parameters. Such latent models may enable more accurate trajectory reconstruction, flexible generalisation across task conditions and the synthesis of novel but valid trajectories within the learnt distribution. This chapter explores robot trajectory planning using CNN-VAE and GAN models. It begins by introducing the proposed architecture, detailing its design, training process and evaluation methods. The chapter then progresses to a disentanglement model that builds upon a similar architectural framework. Following the presentation of both models, the chapter provides comprehensive results from various experiments and concludes with an in-depth discussion. Key contributions in this chapter includes

- A CNN-VAE model for robot trajectory planning by effectively modelling temporal dependencies.
- A GAN-based model improves implicit modelling of complex robot trajectory patterns.
- GAN inversion methods enable identification of the nearest neighbour to a generated trajectory in the dataset, applicable to other domains through two distinct approaches.
- A disentanglement model with CNN architectures facilitates variation of underlying true factors in robot trajectory data.

5.2 Deep Convolutional Neural Networks

A deep Convolutional Neural Network, drawing inspiration from the complex architecture of the visual cortex in animals [307], represent a class of deep learning models specifically designed for processing structured grid data such as images and videos [308]. The primary architectural element in deep CNNs is the convolutional layer, which utilises filters to systematically scan and extract local patterns from the input data. Stacked convolutional layers, commonly complemented by pooling layers, enable these networks to gradually acquire hierarchical representations, abstracting more complicated characteristics from the raw input. Deep CNNs are defined by their depth, with state-of-the-art models having dozens or even hundreds of layers. CNNs have transformed computer vision tasks, setting new records in face detection [309], human pose estimation [310], image classification [311] and semantic segmentation

[312], as well as finding applications in fields other than vision, *viz.*, audio processing [313], video processing [314], autonomous robotics [315], time series data [316] etc.

Deep Convolutional Neural Networks take influence from the neocognitron model [317], adding the notion of a local receptive field, where neurons are sparsely coupled to a limited section of the input data. The foundation of CNNs resides in backpropagation, a method that repeatedly modifies the model's weights or parameters, depending on the error between the expected and actual output. Convolutions are a critical procedure in which kernels (also known as filters) are convolved with the input data, extracting important characteristics. Mathematically, a 1D convolution operation between an input sequence X and a kernel K can be described as:

$$S(i) = (X * K)(i) = \sum_{m=0}^{M-1} X(i+m) \cdot K(m) \tag{5.1}$$

Where:

$X(i)$ is the input sequence at position i with size N.

$K(m)$ represents the kernel (or filter) of size M.

$S(i)$ is the resulting feature map at position i.

To minimise dimensions and computation, strided convolution is utilised. Strided convolution reduces spatial dimensions while simultaneously learning feature representations [318]. This operation modifies the standard convolution by skipping input elements, with the stride value controlling the step size. By using strided convolution instead of pooling, the model achieves both dimensionality reduction and feature extraction, allowing it to capture more complex patterns without introducing non-learnable operations such as max or average pooling. A non-linear activation function, such as the Rectified Linear Unit (ReLU) [319] is employed to incorporate non-linearity into the model. Furthermore, parameter reuse in CNN minimises the number of parameters and decreases overfitting. The concepts of standard and transposed convolution are illustrated in Fig. 5.1, where the effects of convolution with zero-padding (Fig. 5.1a) and transposed convolution (Fig. 5.1b) on a one-dimensional input are depicted.

While convolutional and pooling layers effectively capture local and global patterns, the introduction of dense layers at the end of the CNN architecture plays a

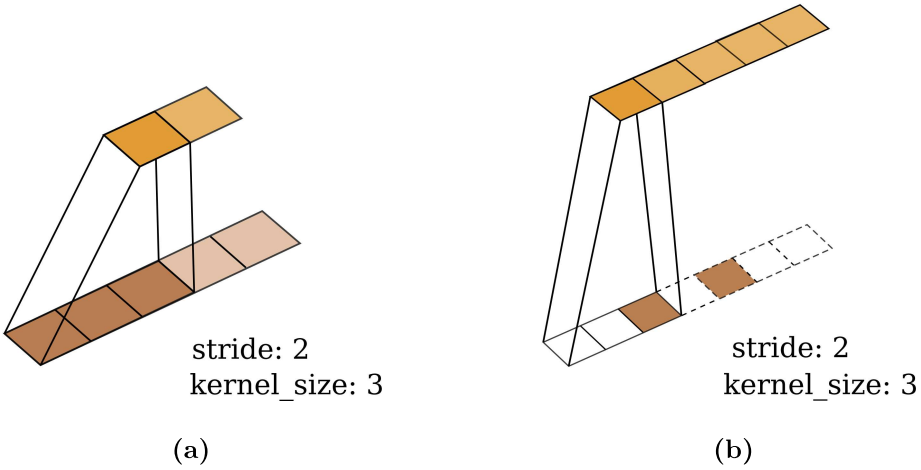


Fig. 5.1: Illustration of convolution and transposed convolution operations applied to a one-dimensional input. (a) Standard convolution with zero-padding and a stride of 2, using a kernel size of 3. A one-dimensional input of length 5 is processed, resulting in an output of length 2. (b) Transposed convolution (also known as deconvolution) with a stride of 2 and a kernel size of 3. A one-dimensional input of length 2 is transformed, generating an output of length 5.

vital role in feature learning. Deep CNN architectures often consist of a succession of convolutional and pooling layers that gradually extract low to mid-level features from the input data. Dense layers also known as fully connected layers, allow the network to learn non-linear combinations of characteristics retrieved by preceding layers. Each neuron in a dense layer is linked to every neuron in the previous layer, which implies that it may examine input from the whole feature space. Mathematically,

$$y_i = f(\sum_{j=1}^{n} w_j x_j + b) \tag{5.2}$$

where,

y - Output vector

w - Weight term.

x - Input vector

b - Bias term

f - Activation function

n - Total number of inputs

Building on this foundation, CNNs have been effectively combined with generative models such as VAEs and GAN to tackle complex learning tasks. CNN-based VAEs leverage convolutional layers for feature extraction while encoding the input into a compact latent space, enabling smooth interpolation between data points and efficient reconstruction of high-dimensional data. Similarly, CNN-based GANs employ convolutional architectures within the generator and discriminator, allowing them to capture sequential correlations in data while synthesizing realistic samples. These hybrid models extend the capabilities of CNNs beyond supervised learning tasks, offering robust solutions for trajectory generation, image synthesis and other applications requiring generative modelling.

5.3 CNN VAE based Learners for Robotic Trajectory Planning

A Convolutional Variational Autoencoder is a type of variational autoencoder designed for conditional generation tasks [320–322]. CNN-VAEs offer significant advantages by combining the feature extraction capabilities of CNNs with the latent space encoding properties of VAEs. This integration allows the model to automatically learn and compress complex robot trajectory information into lower-dimensional latent variables. The benefit of using CNN-VAE lies in its ability to capture high-level features, reduce noise and improve the robustness and generalization of generated trajectories. CNN layers extract sequential or temporal features from the trajectory data, while the VAE framework ensures smooth interpolations in the latent space, aiding in the synthesis of novel and coherent trajectories. In the proposed framework, the input is conditioned on the robot trajectory, with x as inputs, y as labels, z as latent representation. $p(x|z,y)$ represents the prior, $p(z|x,y)$ represents the posterior distributions with normally distributed latent representation $p_\theta(z) = N(z : \theta, I_d)$ where N represents the normal distribution, d is the dimensionality and I_d represents the d dimensional identity matrix.

In this work, a deep learning model is designed to generate trajectories for a robot, where the input is denoted as T representing joint angles $(q_{1i}, q_{2i}, \ldots, q_{mi})$. The trajectory is defined over a duration of $N_{\max} = 128$ time steps, capturing the temporal evolution of the robot's joint configurations. The robot trajectory data

comprises the advancement of time on the horizontal axis and the joint values on the vertical axis. The model is convolved along the horizontal axis to extract features along the joint values over time. The VAE model is created using a stack of 1D CNN layers followed by fully connected layers in the encoder and the fully connected layers followed by convolutional layers in the decoder. The proposed architecture is illustrated in Fig. 5.2.

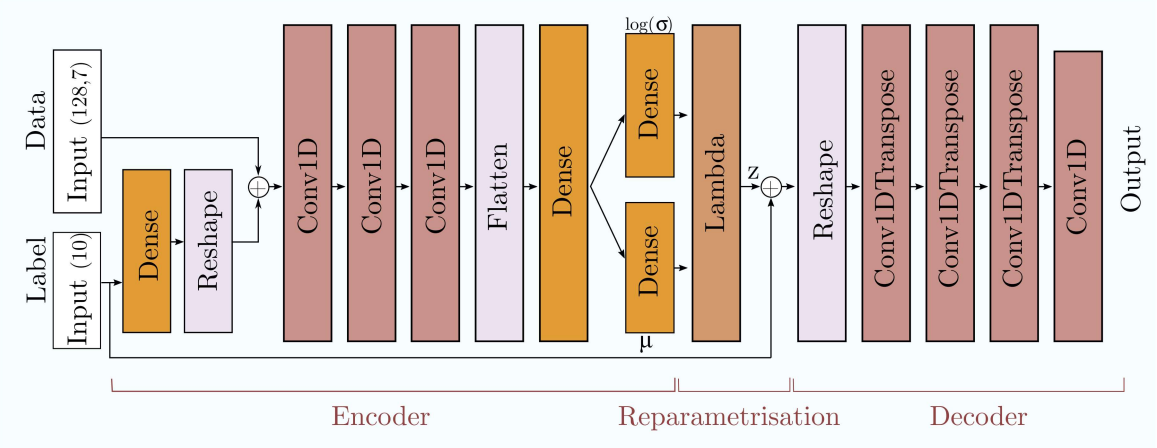


Fig. 5.2: Network architecture of Conditional Convolutional Variational Autoencoder model. Encoder network inputs Data and Label and produce a latent vector. Decoder reconstruct data using latent vector and label.

The input trajectory to CVAE model, denoted as T, is represented as a matrix with dimensions $N_{\max} \times$ timesteps, where $N_{\max}$ corresponds to the maximum number of time steps and timesteps signifies the temporal dimension. The associated labels, indicating the tasks, are represented as Y. Additionally, the latent variable Z belongs to the latent dimension. Mathematically, this can be expressed as $T \in \mathbb{R}^{N_{\max} \times \text{timesteps}}$, $Y \in \mathbb{R}_{\text{task}}$, $Z \in \mathbb{R}_{\text{latent dimension}}$, where $\mathbb{R}$ represents the set of real numbers and $\mathbb{R}_{\text{task}}$ and $\mathbb{R}_{\text{latent dimension}}$ signify the respective spaces for task labels and latent variables. During training, these trajectories are fed into the encoder along with task labels that have been one-hot encoded. The encoder projects this input into a latent vector space, representing the essential features of the joint variables and their respective tasks. Subsequently, the decoder reconstructs the input trajectories based on the latent vector. The training process includes error backpropagation, where the Convolutional Variational Autoencoder loss is a critical component. This loss incorporates the reconstruction loss, quantifying the disparity between the original and

reconstructed trajectories, serving as the joint reconstruction error. The model iteratively refines its parameters by minimizing this loss, thereby enhancing its ability to generate meaningful and task-specific representations in the latent space.

The CNN based generative model is implemented in Python using *PyTorch Lightning*. The computational power for model training and inference is facilitated by an NVIDIA GTX 1050 Ti GPU with 4GB graphics RAM, featuring 768 Compute Unified Device Architecture (CUDA) cores. The overall system is supported by an Intel i7 processor, ensuring efficient processing capabilities for the trajectory generation tasks. A learning rate of $1e^{-4}$ is chosen and the model is trained using a variant Stochastic Gradient Descent optimiser called ADAptive Momentum (ADAM), with a mini-batch size of 128. The dense layer's configuration involves selecting 2048 nodes to ensure effective feature extraction and representation. A dense layer with linear activation function is utilised for μ and σ layers, which then pass to a lambda layer for reparametrisation computation and generate a latent dimensional vector z. The decoder layer consists of a fully connected layer followed by a stack of transposed convolutional layers to upsample the data with learnable layers. Convolutional layers perform downsampling tasks and often display higher noise reduction capabilities when compared to transposed convolutions, where they perform upsampling. Due to this nature, a convolution layer is added to the output of the transposed convolution layer to reduce the spurious artefacts in the output. Equation 3.7 represents the general version of VAE loss. The joint space distance metric as discussed in section 3.8 is employed as reconstruction loss $\mathcal{L}_R$, which can be mathematically represented as,

$$\mathcal{L}_R = \frac{1}{N} \sum_{i=1}^{N} \left\| t_i - \hat{t}_i \right\|^2 \tag{5.3}$$

where N is the total number of trajectories, t_i represents the original trajectory and $\hat{t}_i$ is its corresponding reconstructed trajectory generated by the VAE model.

Additionally, The Variational Autoencoder generates trajectories represented as $\{t_1, t_2, \ldots, t_n\}$, each trajectory t_i comprising joint angles $(q_{1i}, q_{2i}, \ldots, q_{mi})$. To evaluate these joint space trajectories, kinematics is employed using the *PyTorch kinematics* package. The kinematics operations are performed at the tensor level using URDF file, treating the robot as a kinematic chain. This involves the conversion of

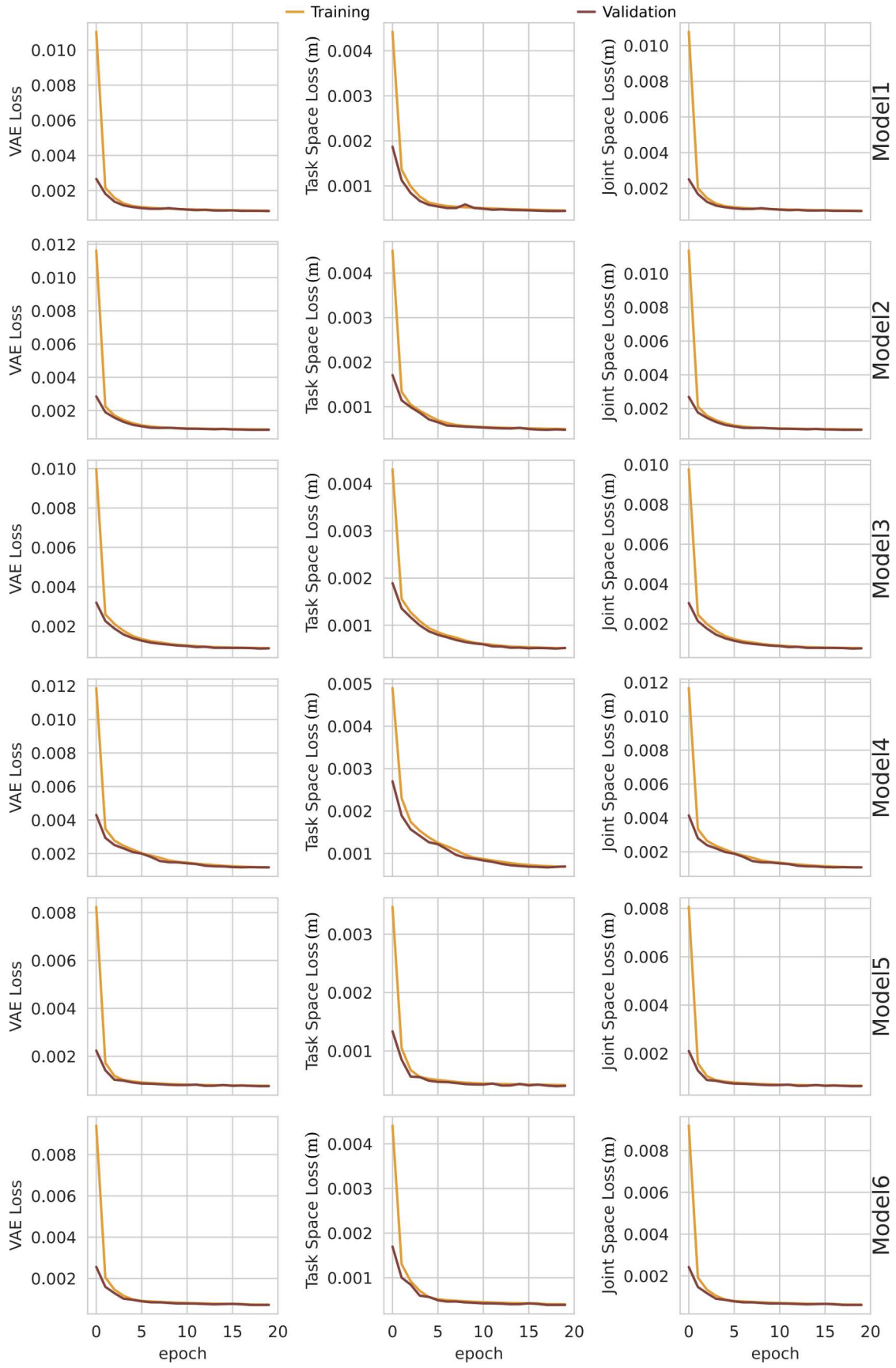
kinematic transformations for precise analysis and representation of the robot's joint configurations throughout the generated trajectories. The kernel size and number of layers are chosen as the hyper-parameter for the model for evaluation. Different hyper-parameters considered for the model are listed in Table. 5.1.

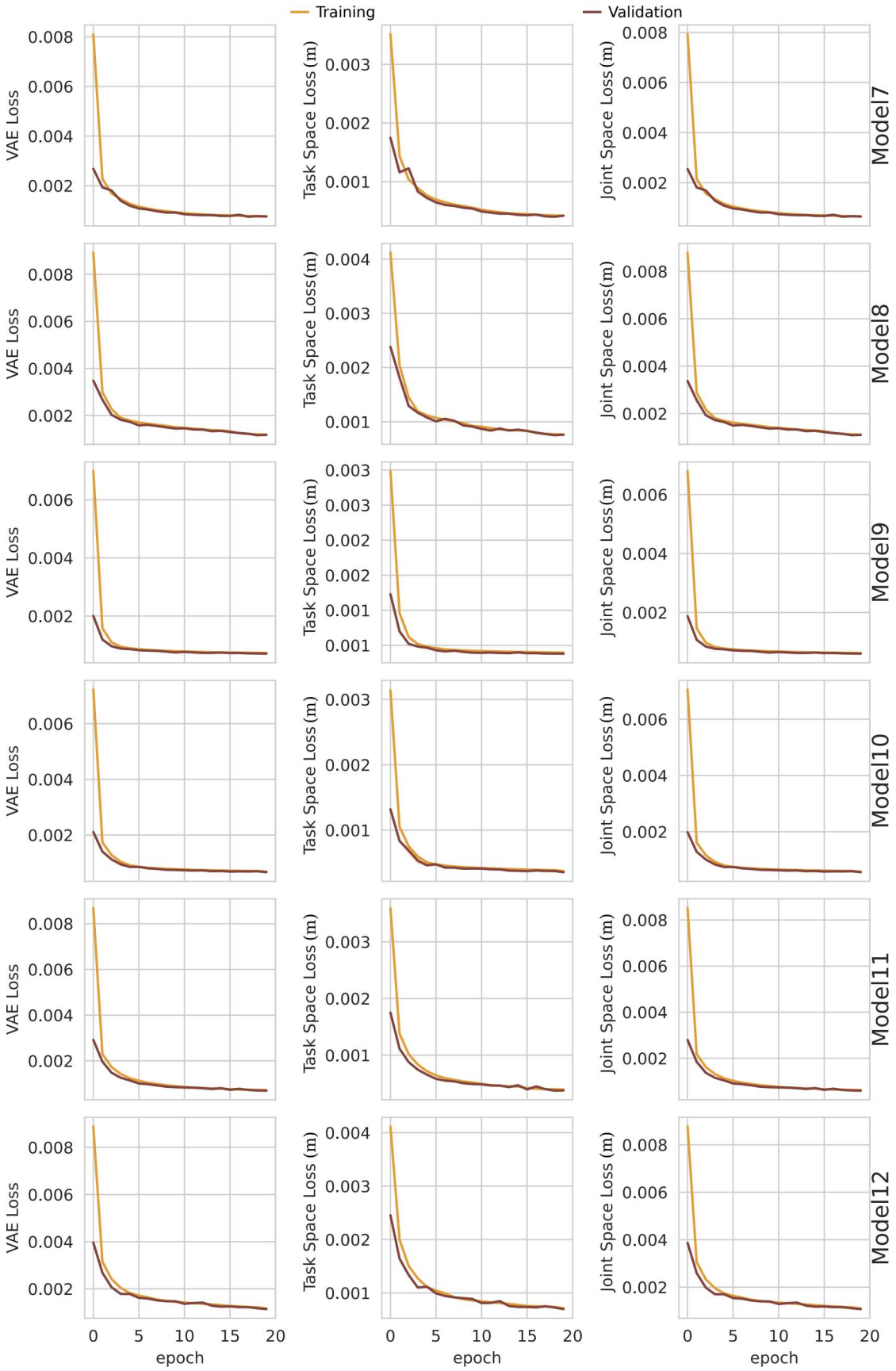
Table 5.1: The hyperparameters of models under consideration for CNN VAE model

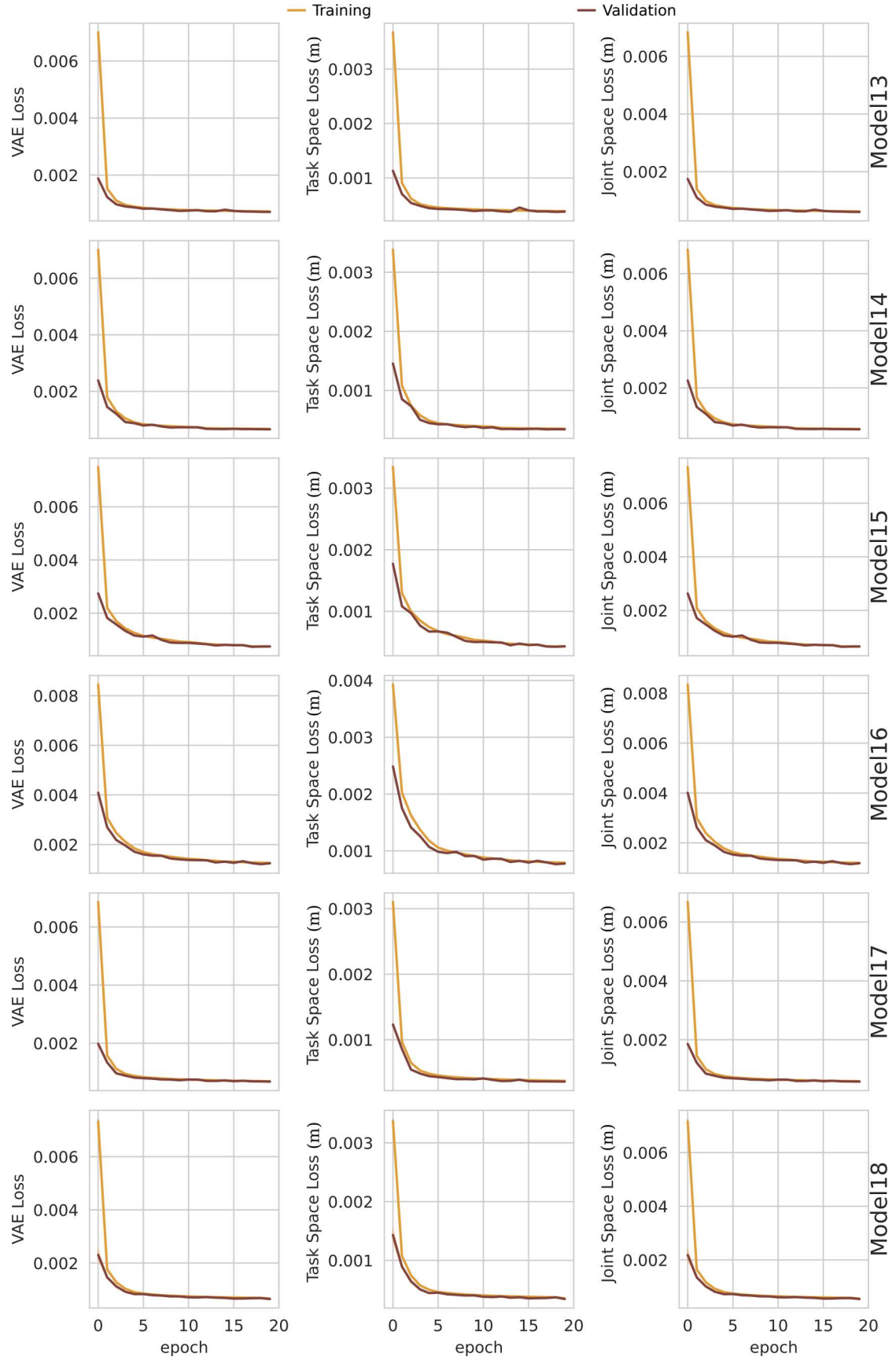
Model	Kernel Size	No. of Layers
Model 1	3	3
Model 2	3	4
Model 3	3	5
Model 4	3	6
Model 5	3	7
Model 6	5	3
Model 7	5	4
Model 8	5	5
Model 9	5	6
Model 10	5	7
Model 11	7	3
Model 12	7	4
Model 13	7	5
Model 14	7	6
Model 15	7	7
Model 16	9	3
Model 17	9	4
Model 18	9	5
Model 19	9	6
Model 20	9	7

5.3.1 Performance Evaluation

The proposed models, as defined by the specified hyperparameters in Table 5.1, were subjected to a thorough evaluation over 20 epochs using a dataset of robot trajectories. The dataset comprises records of joint variables defining robot trajectory sequences over a duration of time. The resulting performance was visually depicted through the evolution of VAE loss in Fig. 5.3, while a comprehensive overview of the combined plots are presented in Fig. 5.4. The evaluation is aimed to assess potential overfitting or underfitting, the adequacy of learning rates and the model's generalization capabilities. Five models (6, 10, 14, 15 and 20) experienced early stopping,


Training
Validation
VAE Loss
Task Space Loss (m)
Joint Space Loss (m)
Model1
Model2
Model3
Model4
Model5
Model6
epoch
0
5
10
15
20


Training
Validation
VAE Loss
Task Space Loss (m)
Joint Space Loss (m)
epoch
Model7
Model8
Model9
Model10
Model11
Model12


Training
Validation
VAE Loss
Task Space Loss (m)
Joint Space Loss (m)
Model13
Model14
Model15
Model16
Model17
Model18
epoch
0
5
10
15
20

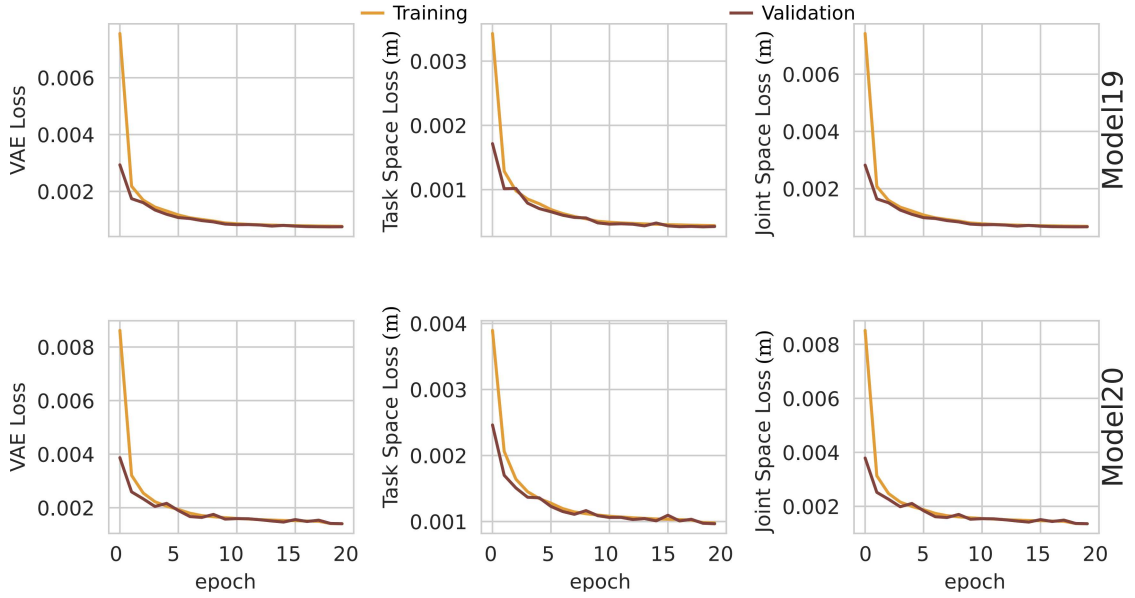
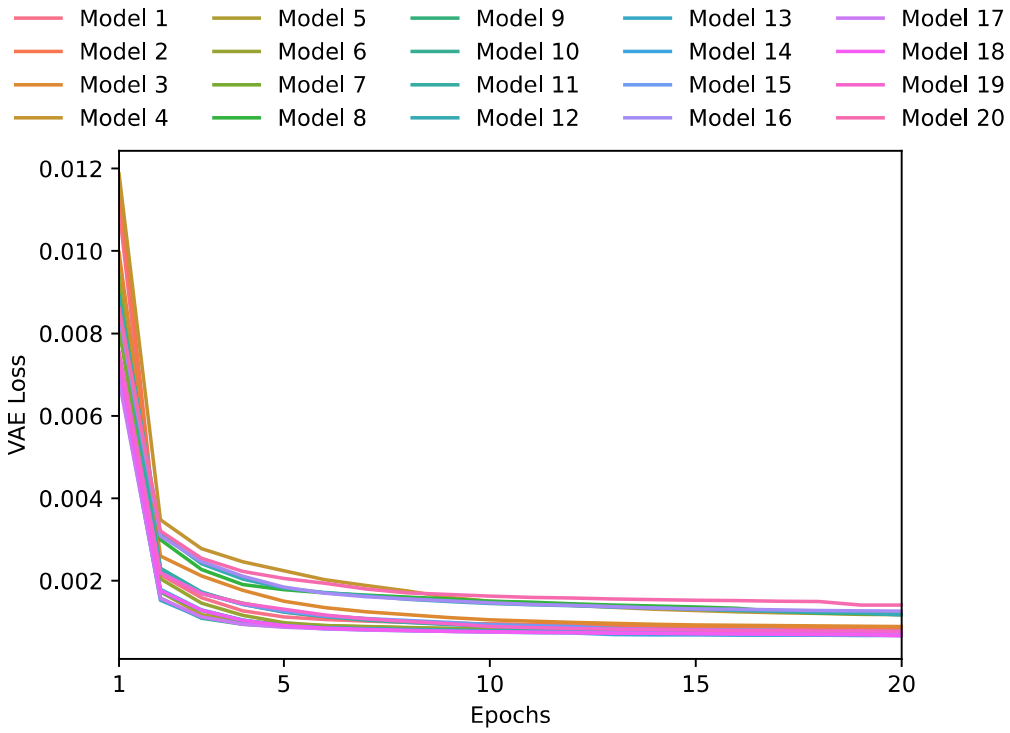


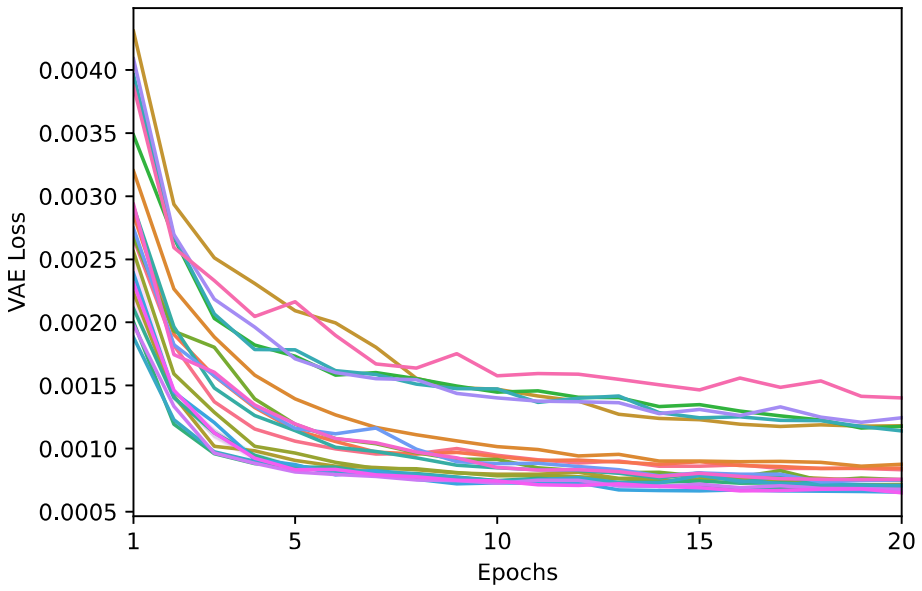
Fig. 5.3: The evolution of VAE loss, Task Space Loss and Joint Space Losses of models described in Table 5.1.

suggesting that they achieved peak performance prior to completing all 20 epochs. The remaining models completed the full training cycle, with their validation loss closely mirroring the training loss trend. This demonstrates effective learning from the training data, suggesting these models successfully identified the underlying patterns without simply memorizing noise. This flexibility in training length, thanks to early stopping, showcases the model's ability to adapt and avoid overfitting, ultimately delivering a diverse set of well-trained models for robot trajectory analysis.

All VAE models were originally set to run for a maximum of 20 epochs, where judicious observation of their loss curves revealed a more nuanced picture. Five models, demonstrating exceptional learning efficiency, reached optimal performance before the full 20 epochs, triggering early stopping. This careful analysis of individual loss trends allowed to avoid unnecessary training time while ensuring all models captured the underlying patterns in the robot trajectory data effectively. This approach not only saved computational resources but also yielded a diverse set of well-trained models, each stopping at the point of optimal learning, avoiding the potential pitfalls of overfitting and memorization. In essence, the selection of the number of epochs became dynamic and data-driven, reflecting to maximizing model performance and efficiency.



(a)



(b)

Fig. 5.4: The combined representation of VAE loss evolution for various models during hyperparameter optimization. a) training loss (b) validation loss.

A detailed analysis of various metrics with hyperparameters kernel size and number of layers is illustrated in Fig. 5.5. The accuracy of the robot path is evaluated using the validation and test datasets with the hyperparameters kernel size and number of layers as shown in Fig. 5.5a and 5.5b. Models with kernel sizes of 7 and 9 generally exhibit the lower average validation path accuracy and test path accuracy. This suggests that a wider receptive field captures richer sequential or temporal context, potentially leading to more accurate path predictions. Kernel sizes of 3 consistently yield the lowest average accuracies for both validation path accuracy and test path accuracy. This trend aligns with the notion that a narrow receptive field may limit the model's ability to grasp the broader context of the path. On the other hand, models with 4 hidden layers demonstrate the highest average accuracies for both validation path accuracy and test path accuracy. This suggests that a moderate layer count strikes a balance between model complexity and representational power, allowing it to learn the underlying patterns in the data effectively. Both very low and very high hidden layer counts (3 and 7, respectively) tend to underperform compared to the middle ground. This is due to underfitting with fewer layers or overfitting with too many layers, highlighting the importance of finding the optimal configuration for the specific task and dataset. The strong positive correlation between validation path accuracy and test path accuracy (0.998) is a promising sign. It suggests that the models generalize well, meaning they can perform accurately on unseen data similar to the training data. This reinforces the value of these findings for real-world path prediction applications. The task space loss across all tasks is evaluated using validation and test losses. The impact of hyperparameters, including kernel size and the number of layers, is analysed to determine their influence on model performance. Similarly, the joint space loss is examined across all tasks to identify trends in path accuracy. Models with kernel sizes of 7 and 9 exhibit superior performance, whereas those with a kernel size of 3 consistently yield the lowest average accuracies. Furthermore, models incorporating 4 hidden layers achieve the highest average accuracies, indicating that an excessive or insufficient number of hidden layers negatively affects performance. This analysis highlights the significance of optimizing hyperparameters to enhance accuracy in both task and joint space representations. Based on these

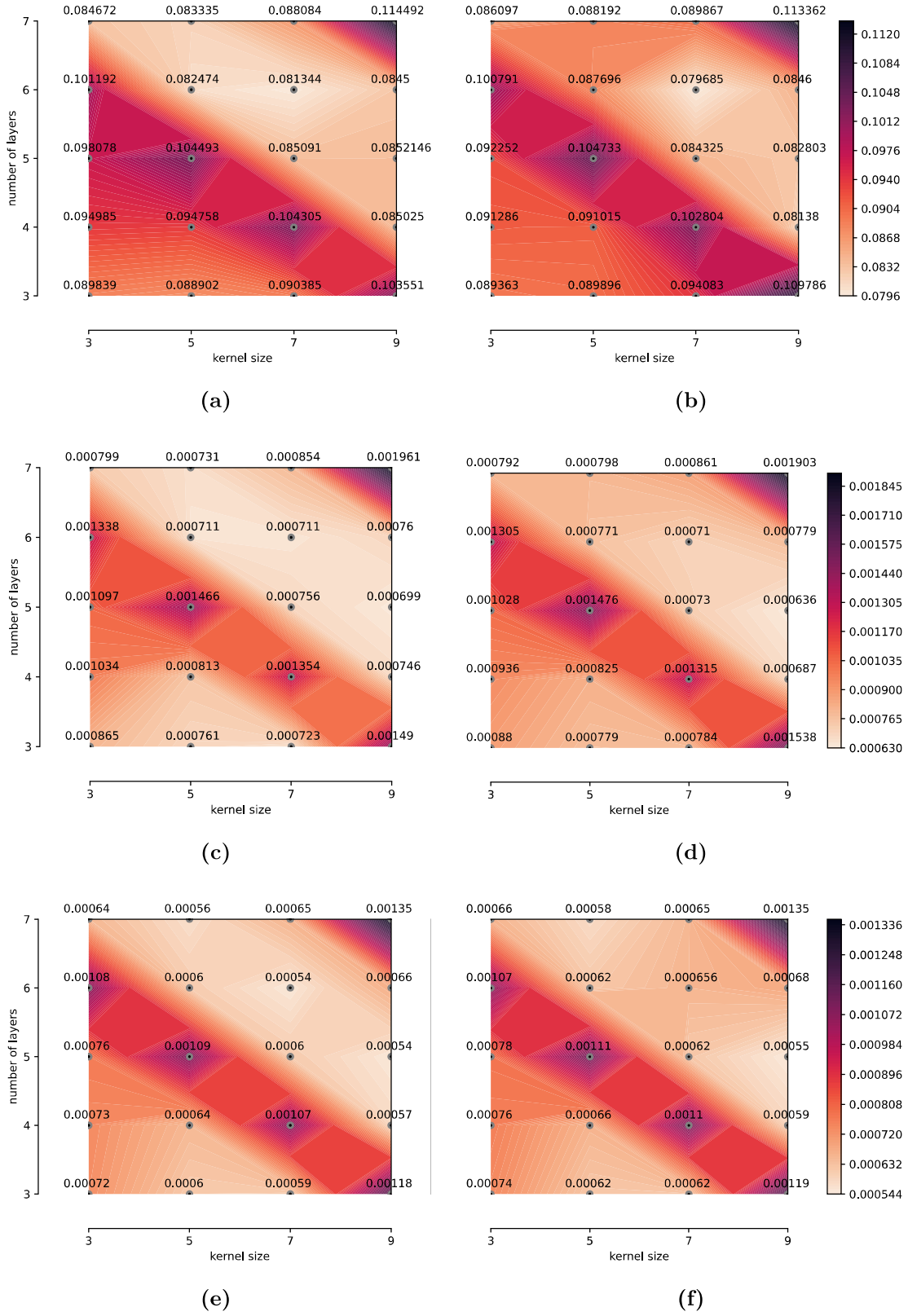


Fig. 5.5: In-Depth Contour Analysis of CNN-VAE Model Metrics. This figure shows contour plots of CNN-VAE model metrics for different kernel sizes and layer counts. The six subplots represents: Path position accuracy (m) for a) validation and b) test datasets; Task space loss (m) for c) validation and d) test datasets; Joint space loss (rad) for e) validation and f) test datasets.

evaluations, Model 14 is selected for qualitative and quantitative analysis. Fig. 5.5c and Fig. 5.5d present contour plots illustrating task space loss, while Fig. 5.5e and Fig. 5.5f depict joint space loss. These plots reveal how variations in hyperparameter settings influence model accuracy.

The motor neuron activations generated by the Variational Autoencoder decoder for robot trajectory data focus on the first 25 feature maps. The transposed convolutional operations performed by the decoder capture temporal transformations and learned patterns during the generative process. The convolutional layer within Model 14 of the decoder provides insights into how complex sequential features are synthesized. These feature maps reveal the neural representations responsible for encoding and decoding the robot's trajectory. This demonstrates the decoder's capability to reconstruct and generate meaningful representations of the trajectory data. The transposed convolution and convolutional layers play a critical role in this process, highlighting the effectiveness of the VAE in learning and reproducing sophisticated patterns. Fig. 5.6 presents the motor neuron activations generated by the decoder, while Fig. 5.6a-5.6d depict the outcomes of transposed convolutional operations. Fig. 5.6e focuses on the convolutional layer in Model 14, showcasing the synthesis of complex sequential features.

Two qualitative analysis experiments have been conducted in this study. The first experiment evaluates the visual similarity between the command path (input path) with the reconstructed path and the interpolation between two robot paths. The reconstruction performance is visualized in Fig 5.7, where the input command path and the reconstructed path are compared, showcasing the effectiveness of the CVAE in capturing and reproducing complex trajectory patterns. To further understand the model's performance, Fig. 5.8 illustrates joint space plots, visualizing the reconstructed paths. Additionally, the projection of the task space onto a canvas provides insights into the CVAE's ability to capture and represent task-specific nuances. It's important to note that the reconstruction process in CVAE is inherently lossy, introducing some level of noise into the reconstructed paths. This noise, while present, remains within acceptable bounds, allowing the CVAE to effectively balance information compression and reconstruction fidelity. Mathematically, the trajectories

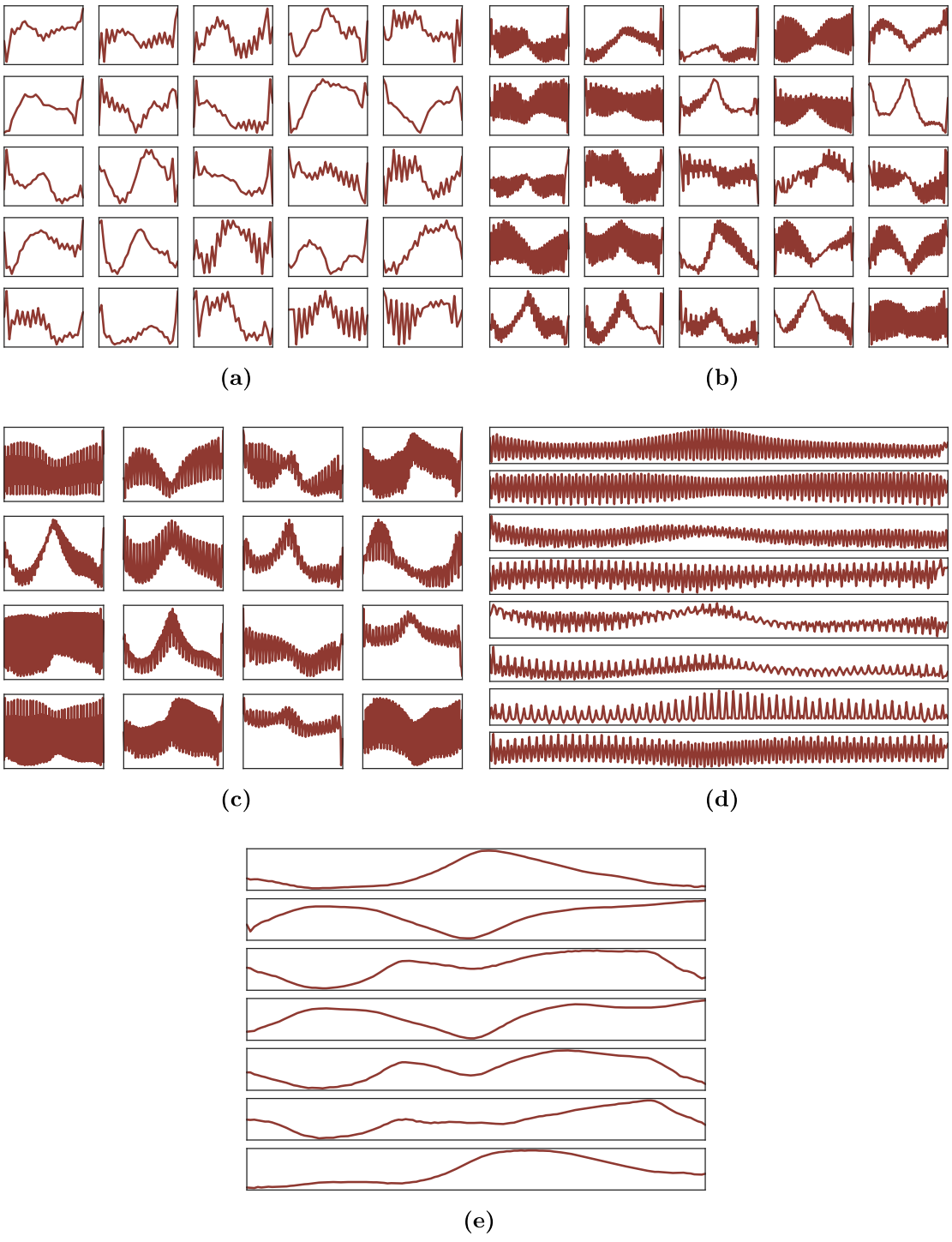


Fig. 5.6: Visualization of the learned feature maps generated by the transposed convolutional layers in the VAE decoder applied to *Task3* in the robot trajectory dataset. *(a)-(d)* illustrates the activation maps of the first 64 filters in the transposed convolution layers and *(e)* represents the output of convolution layer in Model 14 of the VAE decoder, providing insights into the generative process for *Task3* in the robot trajectory dataset.

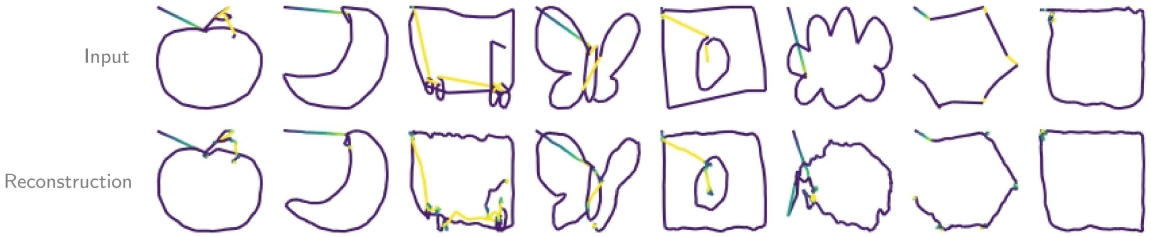


Fig. 5.7: Conditional reconstruction with command pose and the performance evaluation of the models test input from dataset

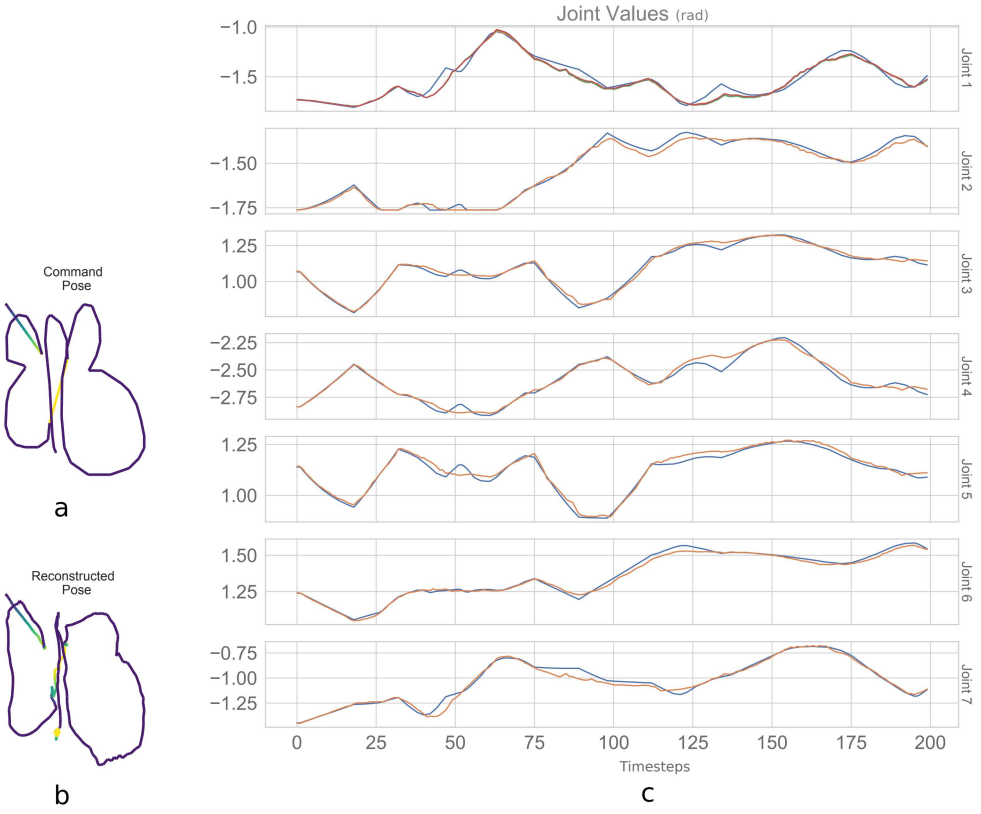


Fig. 5.8: Task space projection shows a) the commanded pose and b) reconstructed x, y pose with a colorbar showing the end effector z pose. c) The corresponding joint angles are shown in figure for all the joints (joints 1 to 7) commanded pose is represented in orange and reconstructed poses are shown in blue

are represented as $\{t_1, t_2, \ldots, t_n\}$ and the reconstruction process can be conceptualized as a mapping function $R : \{t_1, t_2, \ldots, t_n\} \mapsto \{\hat{t}_1, \hat{t}_2, \ldots, \hat{t}_n\}$, where $\hat{t}_i$ represents the reconstructed trajectory. The model learns an abstract representation of robot trajectories in the latent dimension, allowing for compact and expressive encoding of the original trajectories.

One dimensional interpolation in the model's latent space unlocks a powerful toolkit for manipulating and understanding robot trajectories. It ensures smooth transitions between representations, preventing jarring jumps and promoting realistic movement. This interpolation facilitates systematic exploration of the latent space, aiding in the discovery of meaningful variations and contributing to improved generalization for unseen trajectories. The capability to interpolate supports task-specific generation, allowing the model for smooth transition between trajectories associated with different tasks or conditions. Probabilistic latent space interpolation is performed using equation 5.4, where $\alpha \in [0, 1]$, $i, j \in N$, N is the set of natural numbers, z_i and z_j represent two different points on the latent space and z_k represents the linear interpolated representation between z_i and z_j.

$$z_k = (z_j - z_i)\alpha + z_i \tag{5.4}$$

The graphic representation in Fig. 5.9 showcases the linear interpolation technique applied to Task 4, utilizing the CVAE model with two instances extracted from the trajectory dataset. This interpolation mechanism allows for the generation of new

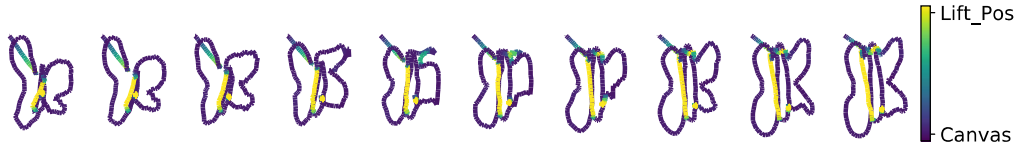


Fig. 5.9: Conditional one dimensional linear interpolation of latent space in Model 14 for *Task4* dataset

latent vectors along a continuous path between z_i and z_j by adjusting the interpolation factor α. This dynamic representation in the latent space can be exceptionally useful for exploring and understanding the latent features that govern robot trajectory data. The advantages of latent space interpolation include its ability to generate diverse

and smoothly transitioning trajectories, providing a nuanced exploration of the latent space's structure.

The pose and path accuracy are evaluated by analysing task space points, focusing on the x, y and z-axis values of commanded and generated poses. The pose position accuracy is assessed, indicating that the values are not uniformly distributed. Most points along the path achieve pose position accuracy below 4 cm and repeatability below 1 cm. The barycentre of the data is positioned close to the commanded pose, demonstrating higher accuracy. Additionally, the generated images exhibit close proximity to each other, indicating improved repeatability. Fig. 5.10 presents the corresponding plots, illustrating task space points with command and generated poses and providing insights into pose position accuracy and repeatability.

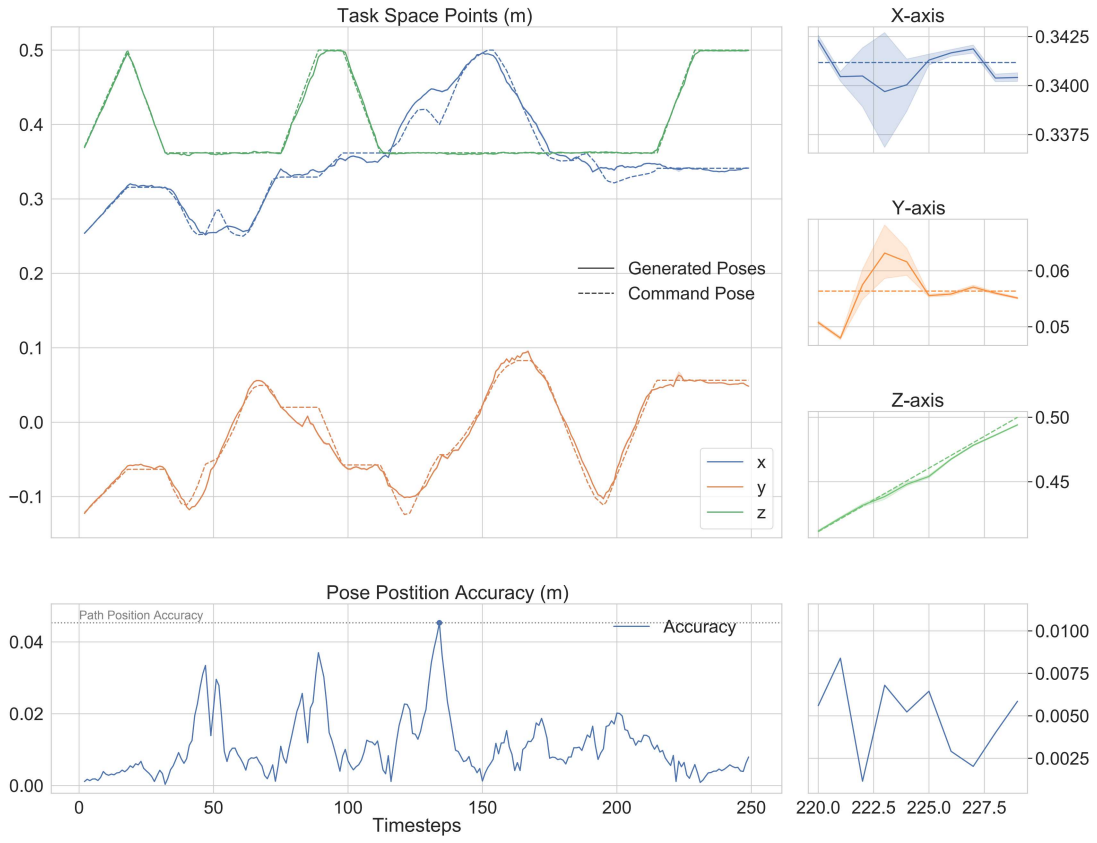


Fig. 5.10: Pose Accuracy of Position of the tip of robotic arm using Model 14.

The taskwise performance of the model is assessed using the path accuracy metric with Model 14. Each task executes fifty different commanded paths in each dataset and the results are evaluated. The best path accuracy of 5.5 cm is achieved

in Task 8, whereas the lowest accuracy of 13 cm is observed in Task 3. This variation in performance indicates that the model's effectiveness varies across different tasks. Factors such as task complexity, data inefficiency and model complexity contribute to this variability. The inconsistency in taskwise performance underscores the need for a deeper understanding of these factors to optimize the model for specific tasks and enhance overall performance. Fig. 5.11 visualizes the taskwise performance, providing insights into the path accuracy metric across multiple tasks.

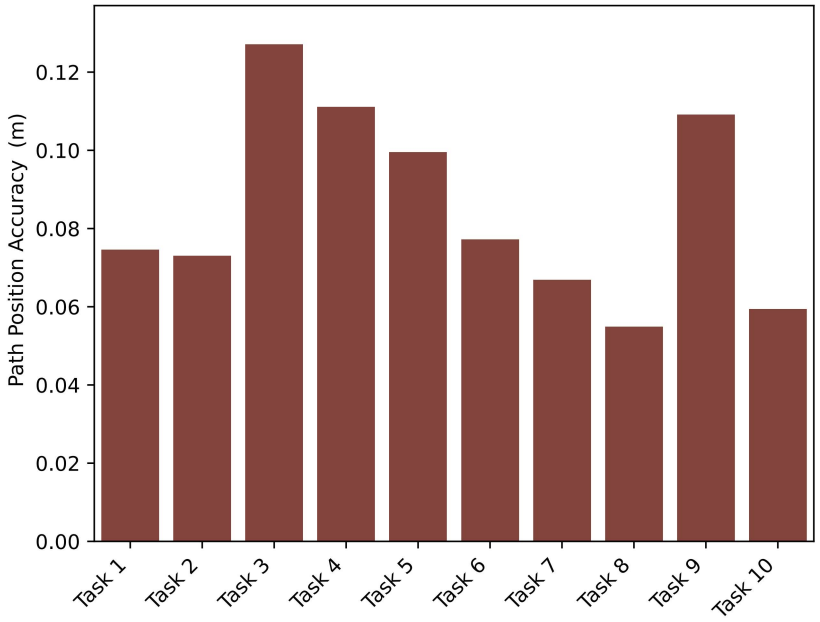


Fig. 5.11: Taskwise Path Position Accuracy of the robot end effector in Model 14. (lower value is better)

5.4 Implicit Sequential Learner for Robot Trajectory Planning with GAN

Spontaneous creation and execution of sequential tasks between players are a daily occurrence in a game. Similarly, creating ordered and sequential dependent trajectory is a challenge for robot trajectory planning. In this context, a strong mathematical framework like game theory allows for the systematic analysis of player interactions as well as the development and evaluation of effective competitive tactics used for problem formulation. The sequential sub-goals of a game is often defined using a Markov Decision Process (MDP) [323] as $\mathcal{M} =< S, \mathcal{A}, R, \mathcal{T}, \tau, p0 >$, where

S denotes the continuous state space, $\mathcal{A}$ represents continuous action space and R denotes the reward function. The transition probability function is represented by $\mathcal{T}$, the time horizon is denoted by τ and $p0$ represents the initial state distribution. Game theory can be utilised to model the strategic interactions between players in complicated contexts. An actor-critic method is a popular approach in the Reinforcement Learning combining policy-based and value-based techniques, which can be modelled as a two-player general sum RL game between the actor (player 1) and critic (player 2) that works well in continuous action and observation space, where both players learn to maximise the reward.

The trajectory consists of a sequence of actions $a_1, a_2, \ldots a_n$, while considering that no unexpected environmental changes will be occurring under a structured environment. Let A_i denotes the set of all possible actions at the ith state; then, the entire action space will be $A = A_1 \times A_2 \times A_3 \ldots A_m$, where m represents the total number of states in the system. Learning-based approaches in the literature learn robot tasks by utilizing various methods. To understand and compare these methods, this work utilizes a game theory approach. The game aims to follow a trajectory to generate actions/tasks and arrive at the Nash equilibrium. The joint action space of size $|A|^m$ is too large for a long sub-goal sequence as it grows exponentially with m. Hence, the convergence of such models in reinforcement learning with continuous action space is exceedingly difficult and computationally costly.

5.4.1 Generative Adversarial Network

In this work, a Generative Adversarial Network [179] architecture is employed in order to directly govern the policy (generator) from raw sequences, which has been proven successful in producing sophisticated higher dimensional data in various disciplines such as graphics, audio and natural language. The model suggests a two-player zero-sum offline game for learning a teacher's (dataset) distribution by a student (player 1) with the help of a judge (player 2). Since the model work under a structured static environment, the judge is able to learn from the environment. The dataset employs only records of robot action trajectory. The defined task has many sub-goals to follow and the learning of higher-dimensional space of such trajectory is implicitly

learned by the model. Two players play a min-max game where one player's gain is another player's loss and the objective function can be represented in Eq. 5.5.

$$\min_{\theta}\max_{\phi} V(G_\theta, D_\phi) = \mathbb{E}_{\mathbf{x}\sim\mathbf{p}_{data}}[logD_\phi(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p(z)}[log(1 - D_\phi(G_\theta(\mathbf{z})))] \quad (5.5)$$

where the G_θ and D_ϕ represent the generator and discriminator network with corresponding learnable parameters θ and ϕ. The objective is to generate the player 1 data following the distributions in the dataset. The generated trajectory becomes relatively better as it eventually reaches the Nash equilibrium. The larger search space in RL can be narrowed by inducing the prior knowledge from data. Here, the model replaces the symbolic prior with the inductive prior to reduce the empirical load.

Furthermore, the work employs actuator sequences and deep generative learning instead of hierarchical reinforcement learning. Despite the fact that these approaches have been proven successful in certain tasks, such as welding or painting along a continuous line defined by a sequence of points, they cannot be separated into sub-tasks. The chapter also employs two methods for computing the neighbours of generated and dataset instances, which are used to compute the robot path accuracy.

The model design, illustrated in Fig. 5.12, employs a GAN model comprising a generator and a discriminator. The generator model randomly samples from the latent dimensional vector $z \in N$ to generate new instances, while the discriminator tries to predict whether a given sample is real or generated. The model is trained

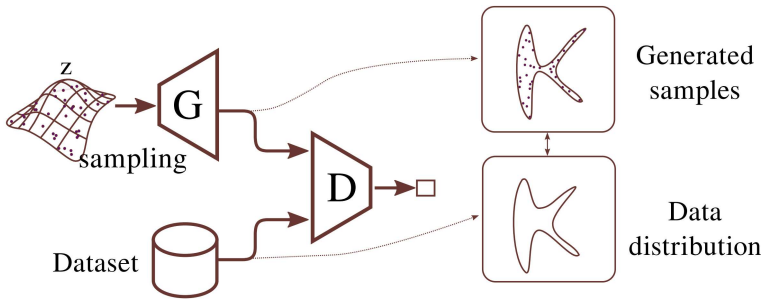


Fig. 5.12: Generative Adversarial Network architecture with the learned and data distributions are shown

with a train dataset with the objective function defined in Eq. 5.5. Eventually, both networks attain a Nash equilibrium where the generator produces realistic trajectories and the learned probabilistic distributions is visualised.

Unconditional GAN model was developed, where each task was trained and evaluated independently on the model. The dataset creation process mirrors that discussed in section 4.6 but involves saving each dataset separately. Table 5.2 lists the datasets utilized in this chapter.

Table 5.2: Generated dataset description with eight categories, each instance is generated and outliers removed

Task	Dataset	Train set	Test set	Total
Task1		49307	12327	61634
Task2		47818	11955	59773
Task3		80000	20000	100000
Task4		44208	11053	55261
Task5		36973	9244	46217
Task6		58488	14622	73110
Task7		73935	18484	92419
Task8		44000	11001	55001

5.4.2 GAN Inversion

GAN inversion aims to employ a pre-trained model with a dataset of robot tasks and their corresponding latent codes since the GAN model does not provide a direct mapping function from the robot task to the corresponding latent representation. This work proposes two methods, which are illustrated in Fig. 5.13 and 5.14.

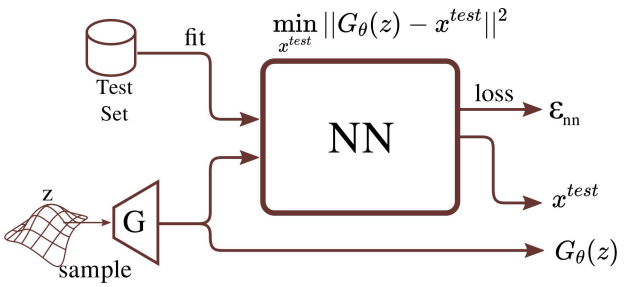


Fig. 5.13: Block diagram of GAN inversion Method I. Generative model sample the latent dimension(z), kNN algorithm finds the nearest trajectory from the test set by minimising the distance function.

The Method I utilises the non-parametric kNN model [241], which learns on a test set with k=2. The most immediate trajectory for a latent vector z is obtained by utilising the projection of vector z into the task space $G_\theta(z)$ with the kNN model. The objective function that reduces the joint space loss and its closest instance from the test set is represented as Eq. 5.6.

$$\min_{x^{test}} ||G_\theta(z) - x^{test}||^2 \tag{5.6}$$

which measures the reconstruction error ($\mathcal{E}_{nn}$) between a generated trajectory $G_\theta(z)$ and its closest trajectory in the test dataset x^{test}. The distance of the generated sample to the nearest true data instance can be considered as a measure of the performance of the implicit generative model.

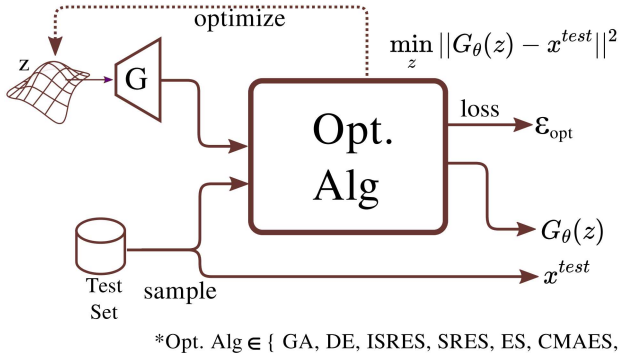


Fig. 5.14: Block diagram of GAN inversion Method II. A trajectory is sampled from the test set, the optimisation algorithm tries to find the nearest possible latent vector, which minimises the distance between generated and test trajectories. A listing of optimisation algorithms denoted by *Opt. Alg*, which encompasses Genetic Algorithm, Differential Evolution, Evolution Strategies, Improved Stochastic Ranking ES, Stochastic Ranking ES, Completely De-randomised Self-Adaptation in Evolution Strategies and Pattern Search.

The Method II employs the optimisation-based inversion method. State-of-the-art meta-heuristic algorithms are best suited for non-differentiable space optimisations. A random trajectory from test dataset x^{test} is selected and computes the nearest latent vector z by utilising the objective function in Eq. 5.7.

$$\min_{z} ||G_\theta(z) - x^{test}||^2 \tag{5.7}$$

5.4.3 Proposed Network Architecture

GAN consists of two models, generator ($G(x)$) and discriminator ($D(G(x))$). Considering the one-shot trajectory planning, the entire trajectory's underlying structure is learned with Convolutional Neural Network layers, which efficiently extract the temporal features from underlying data with receptive fields, weight reuse and local pooling [324].

The generator model employs a 100-dimensional latent vector (z) as the latent space drawn from a uniform distribution and generates samples using a sequence of transposed convolution layers followed by a convolutional layer at the output. A non-linear activation function tanh ($tanh(i)$) is utilised in order to apply a pre-activation for input i. In order to evaluate the performance of the generative model, a discriminator model with one-dimensional convolutional layers is utilized, which captures the input features in data and predicts whether it is real or generated based on the features. A fully connected, intermediate feature layer with a 16384 dimension is employed for evaluating the features for prediction. Specifically, given a sequence $x \in \mathcal{D} = R^{m \times J}$, where J stands for the degree of freedom of the robot along with a further dimension for gripper action control and m represents the fixed duration. The data produced by the generator yields $G(z)$ and the data instance from dataset x contains the same fixed data shape and the discriminator learns to predict the outcome as real or generated. A detailed implemented model of Fig. 5.12 is depicted in Fig. 5.15.

The unsupervised gradient-based sequence generation is carried out using deep CNN models with loss back-propagation. Binary cross entropy loss between the predicted and actual output of the discriminator is employed as the loss function in order to obtain the gradients for training the generator. The model is optimised using a variant of Stochastic Gradient Descent (SGD) called ADAM [325].

5.4.4 Performance Evaluation

The generator network samples latent vectors from a uniform distribution to produce output trajectories. Each vector represents a point in the latent space, which is transformed into a sequence of actions forming a trajectory. The output of the generator varies with each sampling, resulting in distinct trajectory instances. This vari-

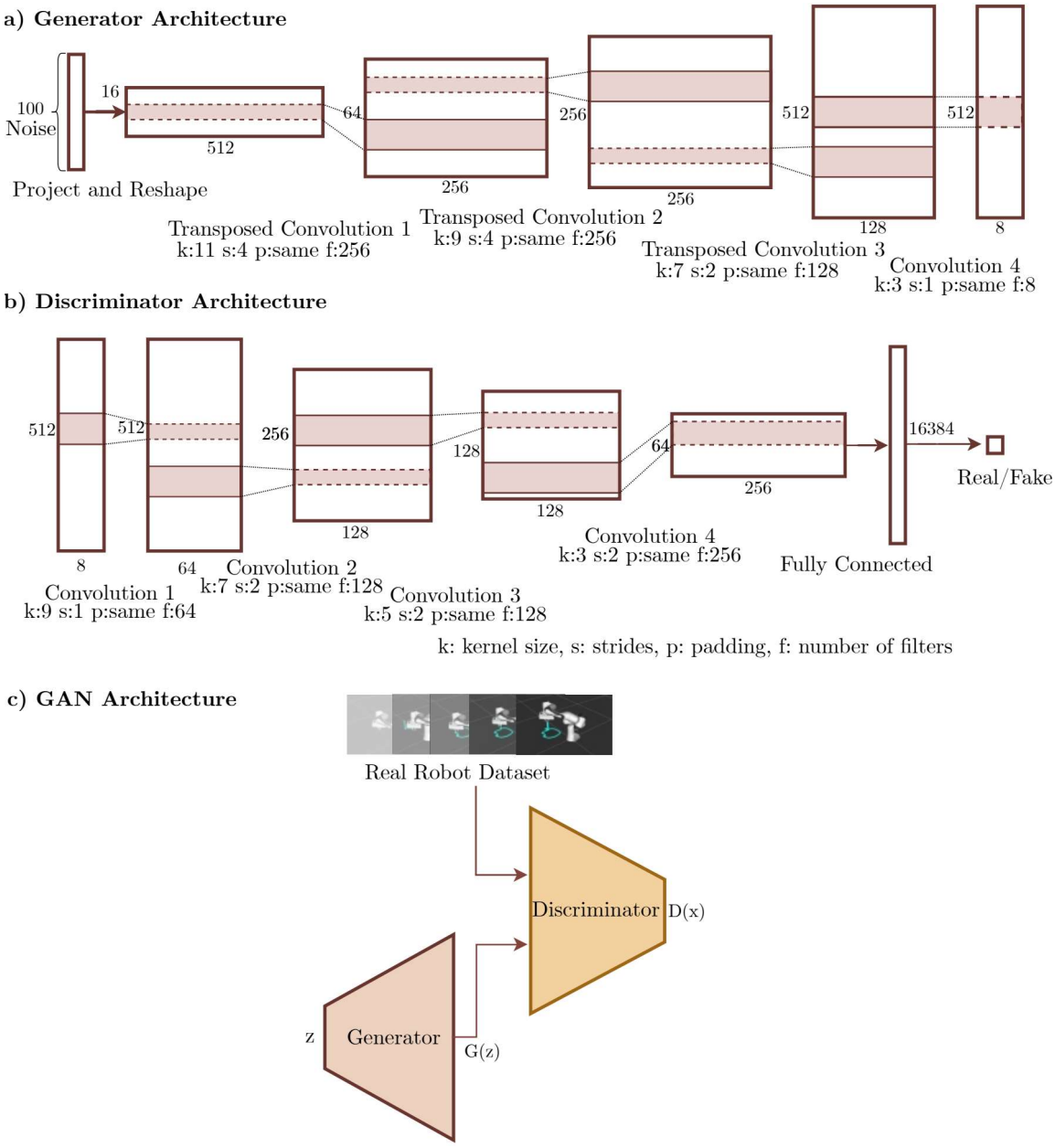
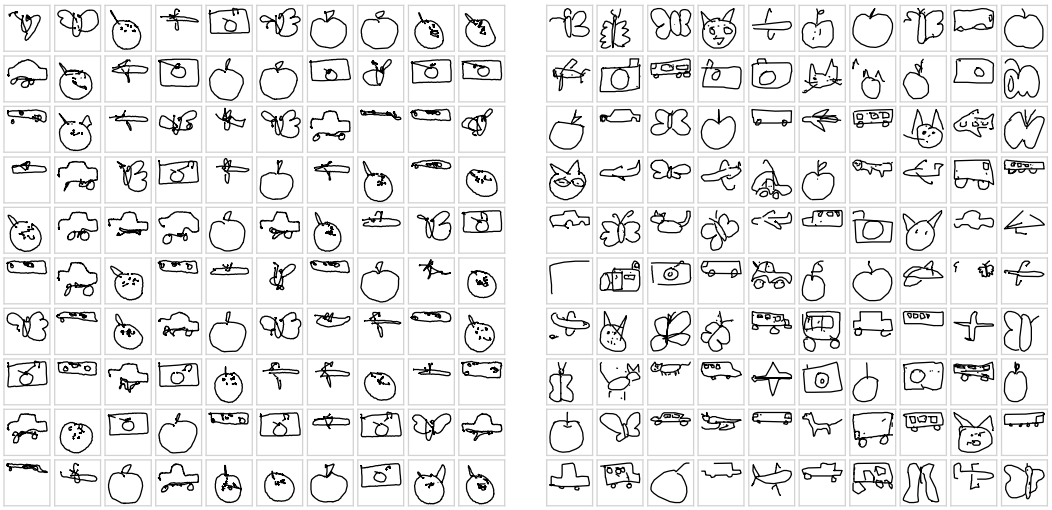


Fig. 5.15: Generative adversarial model architecture consists of convolutional generator and discriminator models. a) Generator model architecture consisting of 100 dimensional latent vector as input with a series of transposed convolution layer followed by a convolution layer as the output. b) Discriminator network with a series of convolutional layers followed by fully connected dense layers and outputs a single vector of label, c) Combined model representation with generator and discriminator models with the dataset input is visualised

ability ensures that the model does not suffer from mode collapse, a situation where the generator produces limited or repetitive outputs[288]. The generated trajectories preserve the semantic structure of the intended actions. However, they exhibit significant variations in stroke order, direction (clockwise or counter-clockwise), length and count. These variations indicate that the generator captures the underlying diversity in trajectory formation. The model is capable of producing trajectories with different sequential and temporal characteristics while maintaining meaningful coherence. The maximum length of the trajectory is limited due to computational constraints. This restriction impacts the ability to generate extended action sequences, especially in tasks requiring long-term planning. A sample of the generated trajectories is illustrated in Fig. 5.16a. The figure shows the diversity of the trajectories produced by the generator. A comparison with the original dataset samples, shown in Fig. 5.16b, reveals that the generated data follows a distribution similar to the real data.



(a) **(b)**

Fig. 5.16: Sample of projections of trajectory on canvas with all categories shuffled. a) Sample of trajectories generated with trained DCGAN model b) Sample of trajectories in the robot dataset synthesised from QuickDraw dataset.

The sample instances from different categories are visualised in Fig. 5.16b. Each instance is of dimension 512×8 where J=8 and m=512. A sample of learned motor drive signals representation is shown in Fig. 5.17. The first 64 learned representations from the initial three generator layers are visualized with the final layer. The generator samples high-level motor drive signals from the random normal distribu-

tion as shown in Fig. 5.17a. Subsequent layers generate trajectory features, illustrated

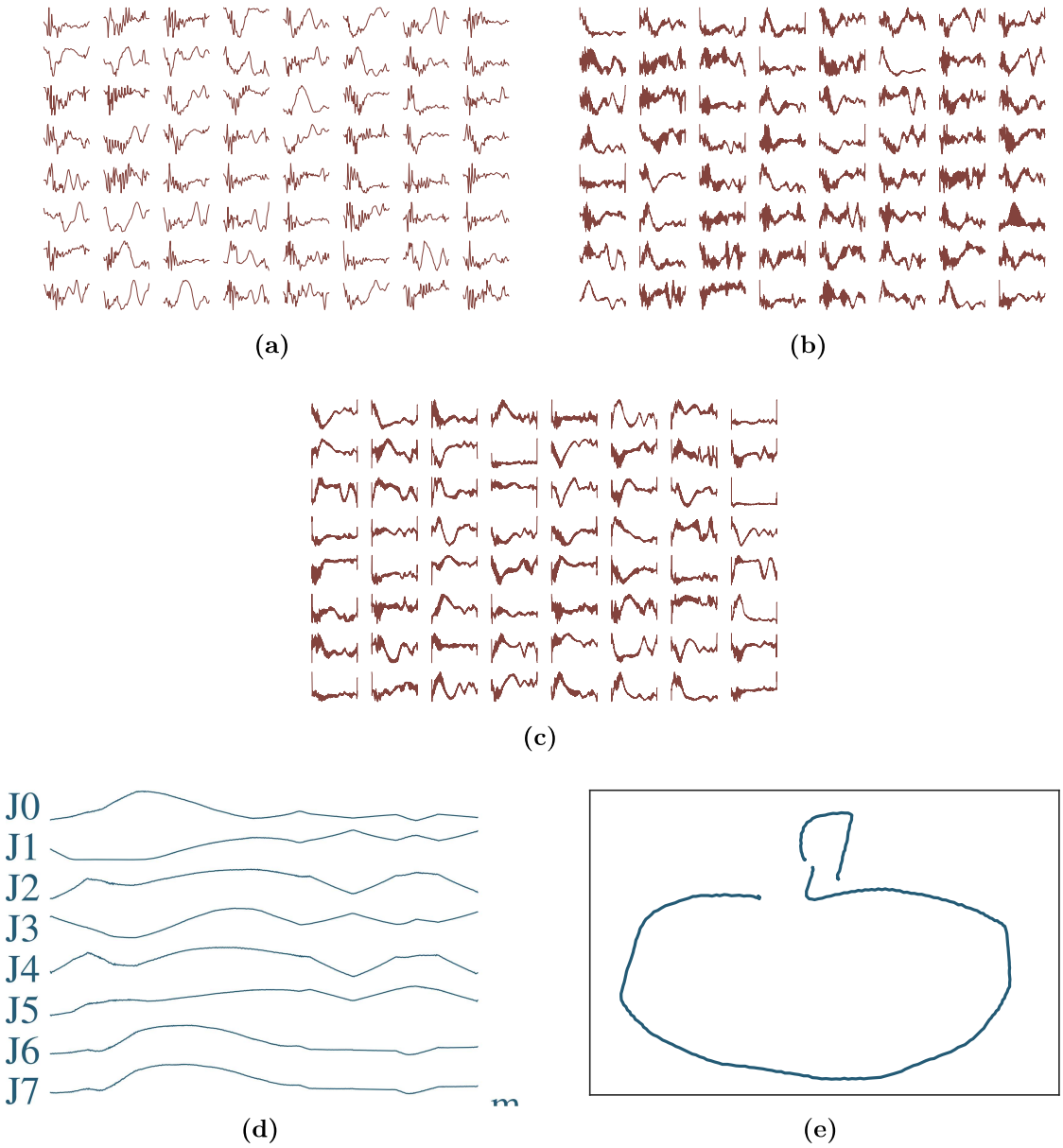


Fig. 5.17: Learned feature maps of generator model from the category *Task7* in the dataset. *(a)-(c)* shows the first 64 filters of transposed convolution layers with dimensions 64, 256, 512 respectively, *(d)* represents the feature map of final convolutional layer and *(e)* represents the output projection on a canvas.

in Fig. 5.17b and 5.17c. A convolution layer has been implemented at the output to downsample the feature space and thus decrease the spurious artefacts to a minimum. From Fig. 5.17d, it can be observed that the output layer features show significant similarities with the original trajectory characteristics and the corresponding trajectory representation on a canvas is demonstrated in Fig. 5.17e.

The model has been trained for 300 epochs to evaluate the network's performance with a batch size of 128. A variant of Stochastic Gradient Descent, ADAM optimisation algorithm has been utilized for a learning rate of 10^{-4} is selected for both discriminator and generator model for all categories. A depiction of basic linear interpolation between two latent dimension vectors is presented in Fig. 5.18, which indicates the feature learning ability of the model from the underlying data representation. The hyper-parameters are optimised through several rounds to guarantee network stability, smooth convergence and prevent mode collapse.

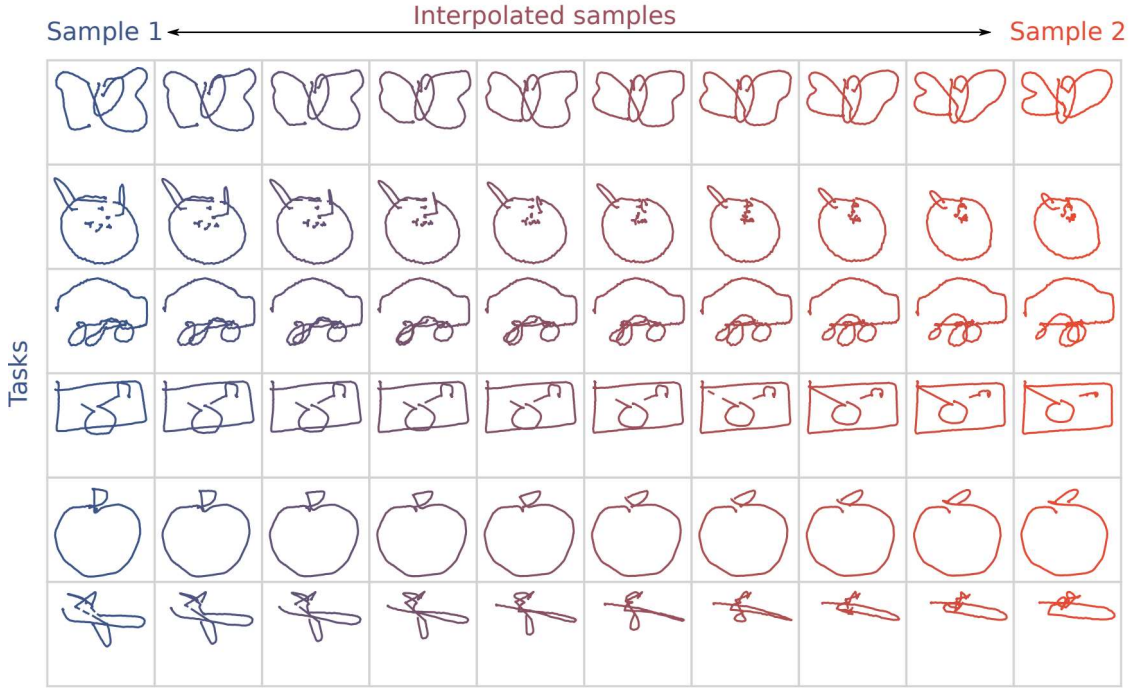


Fig. 5.18: One-dimensional linear interpolated samples in latent space are portrayed with the model trained for 300 epochs.

The fake samples that are generated during the epochs 50, 100, 150, 200, 250 and 300 respectively, are visualized in Fig. 5.19a. The model generates trajectories that aim to replicate the structural patterns observed in the training data. To assess the quality of these generated trajectories, a nearest neighbour analysis is performed. This comparison evaluates the similarity between the generated trajectory and the closest real trajectory in the dataset. Such an evaluation provides insight into the generalization capability of the model. The metric $\mathcal{E}_{nn}$ represents the distance between a generated trajectory and its nearest neighbour from the dataset. A lower value of

$\mathcal{E}_{nn}$ indicates a closer match, implying that the model generates samples that are consistent with real data. The metric α_{path} quantifies the spatial deviation of the generated trajectory from the ground truth in meters. A smaller value reflects higher spatial accuracy. During training, both metrics are tracked across epochs for each task to monitor convergence and performance stability. The metric α_{path} reaches a value of 0.511m, suggesting that the model captures the essential temporal structure of the trajectory with reasonable precision. The progression of the metrics $\mathcal{E}_{nn}$ and α_{path} over training epochs is illustrated in Fig. 5.19b and Fig. 5.19c, respectively. The generated and nearest neighbour trajectories are visualized in Fig. 5.20, providing

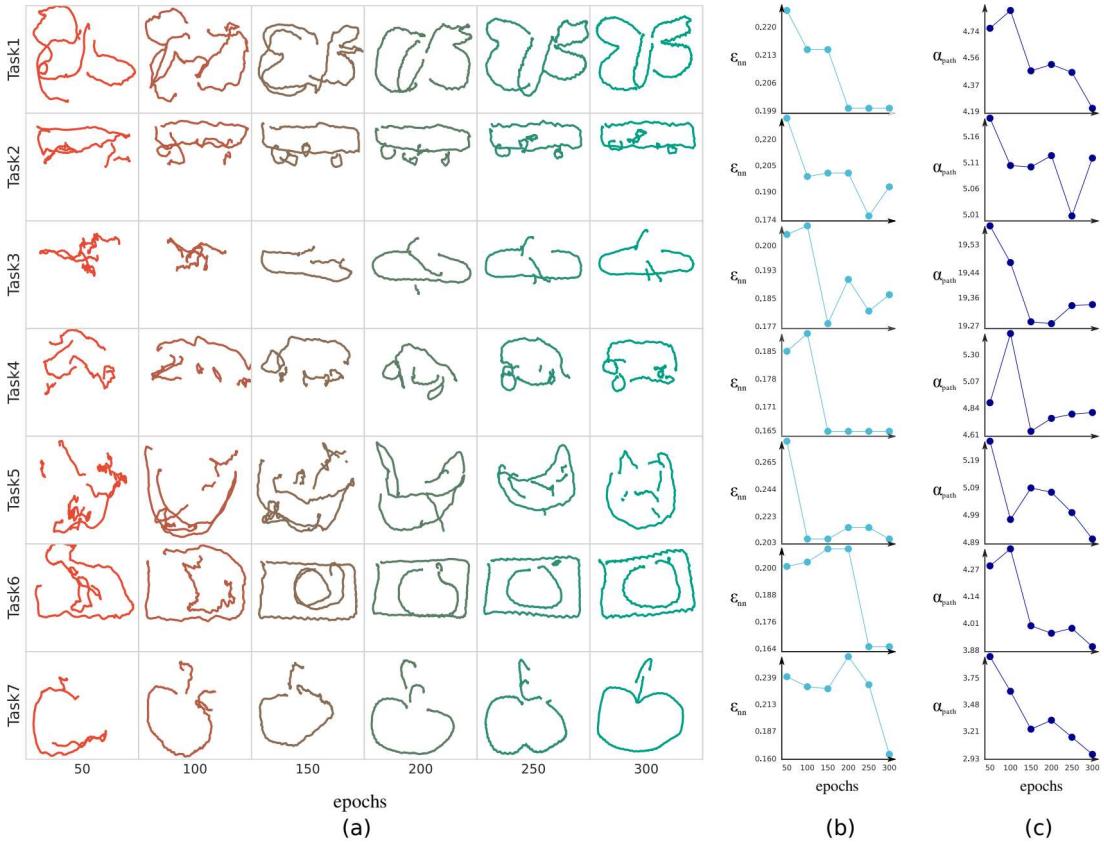


Fig. 5.19: Epoch vs generation of the motor neurons in the model a) trajectory visualisation in cartesian space b) represents the $\mathcal{E}_{nn}$ vs epoch plots and c) represents α_{path} vs epoch plots.

a qualitative comparison. The corresponding numerical results are summarized in Table 5.3.

The reconstruction error, denoted as $\mathcal{E}_{opt}$, quantifies the dissimilarity between the generated and test trajectories. This error metric is minimised during the optimi-

Table 5.3: Comparative Analysis of Optimiser Functions: Robot Path Accuracy (in meters) (Lower is better, Best performance per task is highlighted.)

Category	kNN	CMAES	DE	ES	GA	ISRES	Pattern Search	SRES
Task1	0.094	0.098	0.158	0.110	0.097	0.079	**0.042**	0.114
Task2	0.101	0.106	0.18	0.128	0.108	0.086	**0.037**	0.127
Task3	0.100	0.116	0.167	0.119	0.119	0.106	**0.053**	0.116
Task4	0.127	0.087	0.155	0.099	0.087	0.059	**0.037**	0.108
Task5	0.130	0.126	0.195	0.143	0.128	0.093	**0.044**	0.143
Task6	0.050	0.105	0.157	0.115	0.106	0.089	**0.049**	0.112
Task7	**0.036**	0.109	0.157	0.113	0.113	0.091	0.064	0.109
Task8	0.099	0.101	0.147	0.108	0.092	0.078	**0.063**	0.111
Combined	0.092	0.106	0.164	0.117	0.106	0.085	**0.049**	0.117

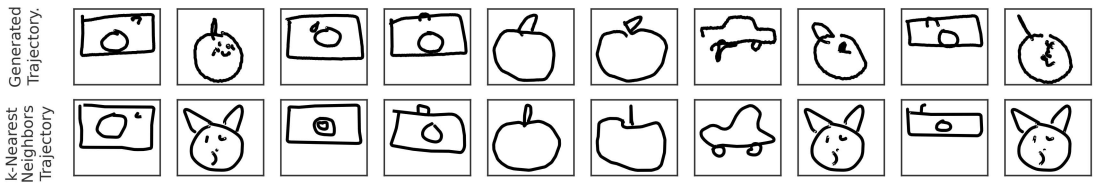


Fig. 5.20: Performance of k-Nearest Neighbors GAN inversion method for a batch of instances. A set of randomly generated instances and the obtained nearest instances from the dataset using nearest neighbouring (Method I) method.

sation process of the latent vector. Several global and local optimisation algorithms are employed to minimise $\mathcal{E}_{opt}$ over 500 iterations. These algorithms include Genetic Algorithm [242], Evolution Strategy [243], Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [244], Differential Evolution [245], ISRES [247], SRES [246] and PS [248]. The optimised error values obtained after convergence for each method are presented in Table 5.3. Among these, Pattern Search exhibits superior performance with significantly fewer iterations. Under identical hyper-parameter configurations, a tenfold reduction in error for α_{path} is observed in Method I compared to Method II. Method I identifies the nearest point of the generated trajectory within the test dataset. In contrast, Method II selects a fixed test instance and executes a search in the latent space to approximate the corresponding trajectory. Fig. 5.21 presents the $\mathcal{E}_{opt}$ values over the optimisation horizon for each algorithm.

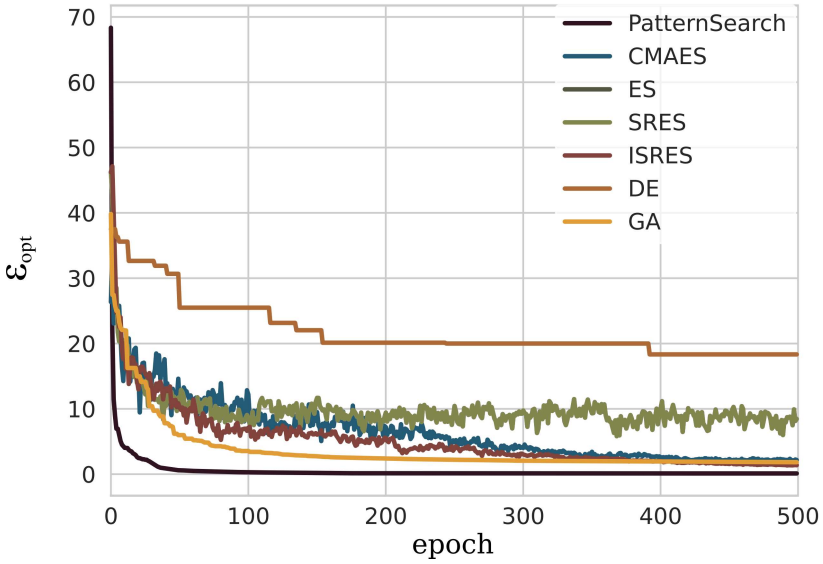


Fig. 5.21: Performance of different optimisation algorithms on an instance of *Task1* subset in the dataset.

A detailed visualisation of the trajectories and metrics is presented in Fig. 5.22. The test trajectory and generated near-optimal trajectory are both visualised in Fig. 5.22a. In order to get a deeper insight into the top-performing models, end effector positions are represented in Fig. 5.22b. Pose position accuracy of the instance is plotted in Fig. 5.22c by computing the barycentre from 20 trajectories and corresponding path position accuracy is plotted in Fig. 5.22d. The joint trajectory values are plotted in Fig. 5.22e. From the joint and task space trajectories, it is clear that the Pattern Search outperforms and generates near-optimal instances as the test trajectory. The Fig. 5.22a provides visual support for this conclusion, showcasing the Pattern Search optimised trajectory and test trajectories. The striking visual similarity between these trajectories underscores the capacity of the proposed Method II to accurately replicate complex motion patterns.

For analysing the performance of a population using optimisation algorithms, a sample size of 20 from each algorithm is chosen and plotted the mean and standard deviation (error bar) values in Fig. 5.23. The class-wise $\mathcal{E}_{opt}$ is plotted in Fig. 5.23a and the α_{path} is computed and plotted in Fig. 5.23b. The cumulative performance of $\mathcal{E}_{opt}$ and α_{path} are visualised in Fig. 5.23c and Fig. 5.23d respectively. The time consumed for each algorithm for 500 iterations is visualised in Fig. 5.23e. The Pattern Search algorithm demonstrates superior performance across all tasks, showcasing its

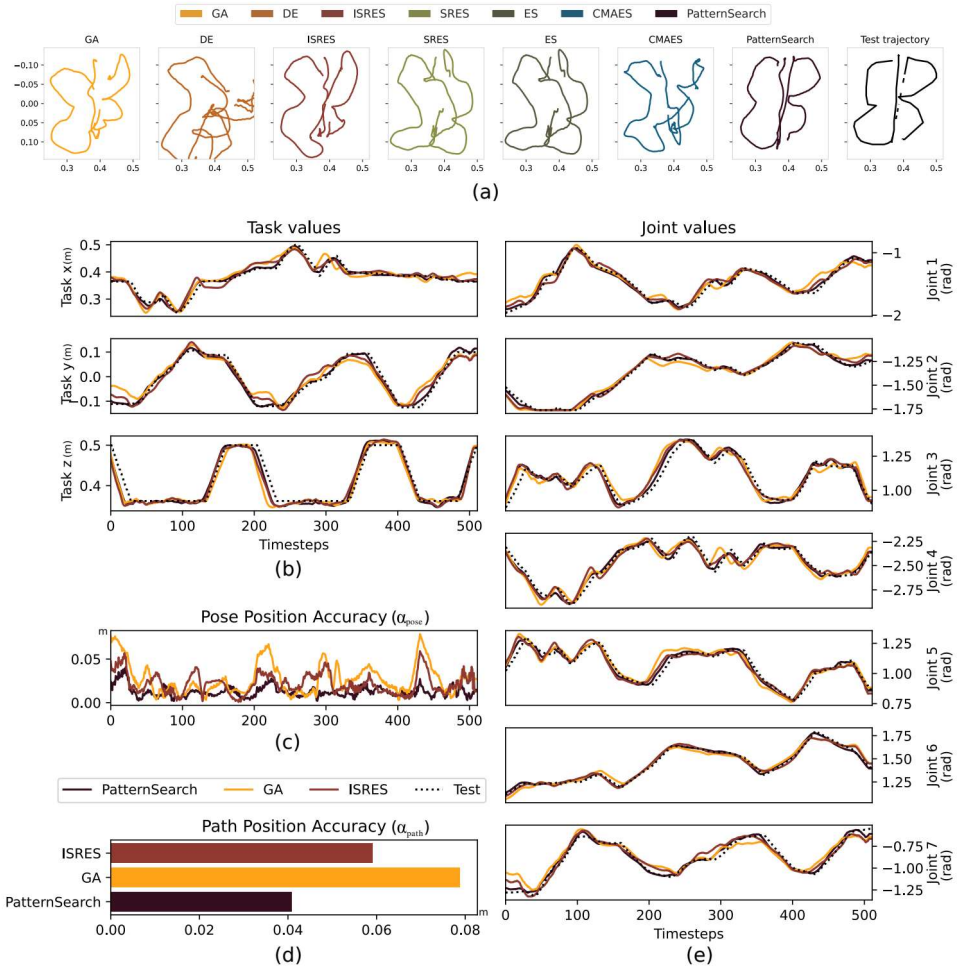


Fig. 5.22: Near optimal trajectory computation for an instance from *Task1* in the test dataset with different optimisation algorithms. a) Trajectory representation of a test instance from *Task1* in the dataset with near optimal representations computed using different optimisation algorithms. b) End effector movement plots of x,y and z axis plots c) Pose position accuracy of robot d) corresponding path position accuracy and e) robot' joint movement analysis with test dataset instance.

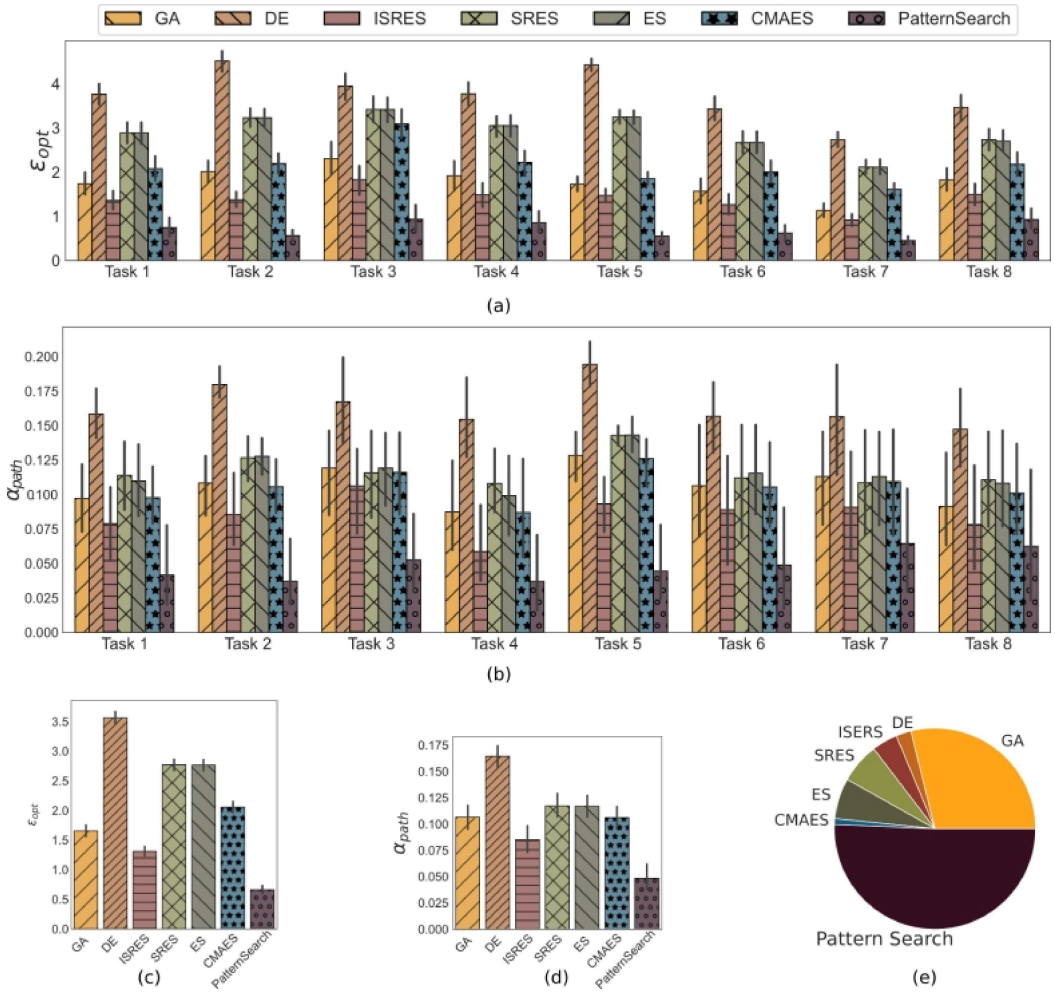


Fig. 5.23: Computed Loss and path position accuracy of different subsets in dataset with different optimisation algorithms in the experiment. a) class-wise loss value plotted with mean squared error b) class-wise path position accuracy (in meter) c) combined loss values d) combined path position accuracy (in meter) e) time consumed for the computation.

effectiveness in diverse conditions. The proposed approach achieves a remarkable combined path position accuracy of 4.8 cm. It is very evident that, even though the Pattern Search performs well, it is time-intensive, which corresponds to much higher computational cost.

5.5 Disentanglement learning using βVAE

Humans possess a remarkable ability to observe and understand the world in a disentangled manner, focusing on features like shape, size and color instead of pixel-by-pixel details. Inspired by this cognitive approach, trajectory planning in the context of CNN VAE is explored, aiming for a disentanglement model. A dedicated disentanglement dataset, utilizing a Selective Compliance Assembly Robot Arm robot [326], is discussed in Section 4.5. This dataset serves as the foundation for the proposed algorithm, developed in Python using the *TensorFlow* deep learning framework [327]. The proposed algorithm conducts kinematics computations using the *Peter Corke toolbox for Python* [328], defining a kinematic chain tailored to the specific requirements of the robot. This interdisciplinary approach integrates cognitive principles, deep learning methodologies and robotics computations to advance the understanding and application of disentangled representations in trajectory planning.

In the proposed disentanglement robot model, the encoder generates the mapping function from lower dimensional raw trajectory data to higher dimensional representation. All the interpretable higher-dimensional vectors generate the mapping function from higher-dimensional representation to lower-dimensional motor signals in the decoder, which encapsulates all the kinematics complexities, sequence, and task information. The model develops a generative model for robot task planning. It learns a human-interpretable, disentangled latent space, which is an effective method for creating new data based on underlying variations.

The proposed network uses a β-VAE model to disentangle robot trajectory representations, with the architecture detailed in Section 3.5.1 and a latent dimension of 10. Although there are only five disentanglement factors, a latent dimension of 10 is chosen to ensure the model captures complex variations in the data. This approach aligns with findings from empirical studies in latent space modelling. The β-VAE loss

(Eq. 3.5.2), controls the balance between reconstruction accuracy and latent space regularization. Selecting the β parameter is critical, as it affects the trade-off between reconstruction quality and factor disentanglement. Higher β values emphasize disentanglement by penalizing the KL divergence term but may degrade reconstruction accuracy. Lower β values (<1) focus on reconstruction at the expense of disentanglement. This work evaluates three β values for comparison: $\beta = 1$, representing a standard VAE, along with $\beta = 5$ and $\beta = 10$. These choices allow for a detailed analysis of the impact of β on disentanglement and reconstruction performance. The optimal β is identified through empirical evaluation using hyperparameter search, latent traversals and disentanglement metrics, considering the characteristics of the trajectory dataset.

5.5.1 Performance Evaluation

In order to evaluate the performance, each model is trained for 100 epochs with a batch size of 128. The resulting latent vector z(codes) is evaluated against the factors in the dataset and different metrics calculated for measuring various disentanglement metrics.

The disentanglement metric has undergone significant improvement with the introduction of the MIG metric. MIG demonstrates substantial enhancement over traditional VAE models by effectively measuring the degree of disentanglement. However, this improvement often comes at the cost of reconstruction performance. Since MIG is not directly integrated with the loss function, its value does not increase monotonically during training. The reconstruction performance of the models is evaluated using various metrics, which are presented in Table 5.4. A comparative analysis of the disentanglement evolution is depicted in Fig. 5.24, while the reconstruction performance of the models is shown in Fig. 5.25, , where Fig. 5.25a shows the input trajectory projection, Fig. 5.25b presents the Variational Autoencoder results, Fig. 5.25c displays βVAE (β=5) performance and Fig. 5.25d illustrates βVAE (β=10) results. These evaluations provide a comprehensive understanding of the trade-off between disentanglement and reconstruction performance."

The work aims to find the trade-off between reconstruction loss and disentanglement. The VAE model achieves a minimum path position accuracy (0.050 m),

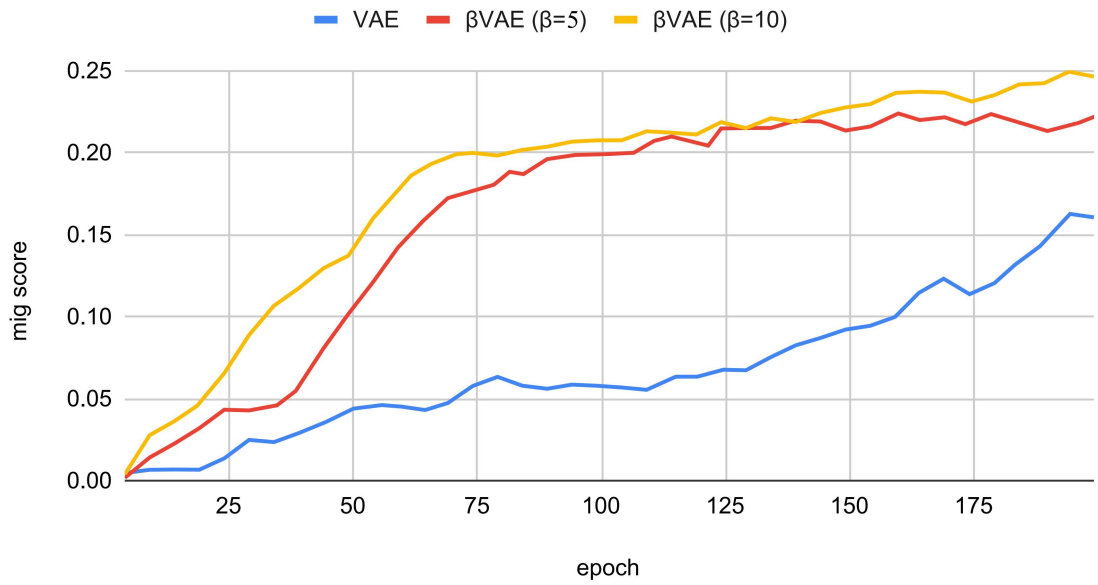


Fig. 5.24: The evolution of MIG score metric for different models

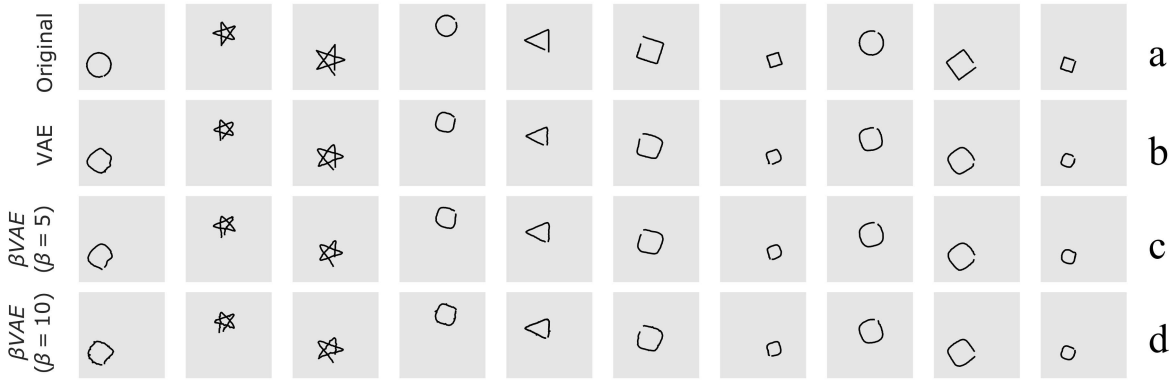


Fig. 5.25: An illustration depicting the reconstruction performance of different models, including a) the input trajectory projection, b) Variational Autoencoder, c) βVAE ($\beta = 5$) and d) βVAE ($\beta = 10$).

Table 5.4: Metrics Considered for Comparing Disentanglement Model Performance. (↑ Means higher is better)

Model	VAE	βVAE ($\beta = 5$)	βVAE ($\beta = 10$)
Path Accuracy ↓ (m)	**0.050 ± 0.015**	0.055 ± 0.032	0.076 ± 0.058
MIG [251] ↑	0.17	0.225	**0.249**
Disentanglement [252] ↑	0.735	**0.980**	0.942
Completeness [252] ↑	0.185	0.241	**0.265**
Informativeness [252] ↑	0.170	**0.248**	0.235
jemmig [253]↑	0.270	0.315	**0.330**
βVAE Score [249] ↑	0.582	**0.644**	0.630
FactorVAE Score [250] ↑	0.625	**0.672**	0.701

Values represents mean± standard deviation

with a better reconstruction performance but poor disengagement (MIG score = 0.17). $\beta\text{VAE}_{(\beta=5)}$ model has a better disentanglement score of 0.225 and a path accuracy of 0.055m. The figure shows that the $\beta\text{VAE}_{(\beta=10)}$ model has a slightly greater MIG score (0.249) than the $\beta\text{VAE}_{\beta=5}$ model, but has a higher path accuracy of 0.076m. For the performance evaluations, models in VAE, $\beta\text{VAE}_{(\beta=5)}$ and $\beta\text{VAE}_{(\beta=10)}$ are considered.

The latent dimensional transversal representation of the best-performing model is depicted in Fig. 5.26, rows 2-11. Each row showed the transversal of only one code. Fig. 5.26a, 5.26b and 5.26c showed the transversal in VAE, $\beta VAE_{\beta=5}$ and $\beta VAE_{\beta=10}$ models respectively with Gaussian reconstruction loss. While considering the VAE model in Fig. 5.26, the 9th row show x position transversal and the 6th row show y position transversal. 10th Row shows the scaling transformations. Other rows produce similar instances over the changes in the corresponding code.

The $\beta VAE_{\beta=5}$ in Fig. 5.26b depicts x position, y position and orientation transversal in the 8th, 3rd and 2nd rows respectively. Scaling is embedded in code in the 3rd , 9th , 10th and 11th rows. The 6th and 7th rows produced shape transversal whereas other rows do not produce much difference in code variations. The $\beta VAE_{\beta=10}$ in Fig. 5.26c produces position x, y and scale transversals in the 4th, 3rd and 6th rows respectively. The 5th and 7th rows embedded code for shape; other rows did not produce any visible difference in the output.



Fig. 5.26: The first row shows the input image and Rows 2-11 show the latent transversal performance of the model with a) VAE model b) $\beta VAE_{\beta=5}$ and c) $\beta VAE_{\beta=10}$ The performance of each model is compared across different codes, with each row representing a specific code and the columns showing the performance of that code while keeping all other codes constant.

While analysing the variations, the VAE model disengagement performance is weak, while $\beta VAE_{\beta=5}$ and $\beta VAE_{\beta=10}$ encode the codes and have achieved a higher disentanglement score. It can be observed that the reconstruction performance is quite prominent in lower β values.

The performance analysis of different disentanglement representations, losses and robot path accuracy are listed in Table 5.4. The higher value of standard deviation in Reconstruction loss, accuracy and repeatability are due to the generative nature of the model. Accuracy and repeatability were calculated in task space and the Reconstruction loss in the joint space of the robot. The non-linear kinematics operation developed variations between the reconstruction values and the robotics metrics. Joint space reconstruction loss is lower in the VAE model. As the β value increases, reconstruction performance decreases. However, there is a permitted limit to each task's precision and accuracy range. optimisation of β based on task nature and required disentanglement can be achieved by hyper-parameter tuning. This work uses the MIG score as the primary metric for evaluating disentanglement.

5.6 Summary

The chapter explores the integration of advanced Generative AI models, specifically Variational Autoencoders and Generative Adversarial Networks, both utilizing 1D Convolutional Neural Network layers, for robot trajectory planning. The studies emphasize optimizing model performance through hyperparameter tuning and evaluating efficacy using metrics such as path accuracy, joint space loss, task space loss and disentanglement scores.

The VAE model, incorporating CNN layers, achieves a maximum path accuracy of 13 cm after rigorous validation, with detailed evaluations of joint and task space losses providing insights into its effectiveness. Similarly, the GAN-based approach, also leveraging CNN layers, tackles challenges in implicit models through two methods: Method I, a nearest-neighbour technique for efficient trajectory retrieval and Method II, which uses Pattern Search optimization, attaining a superior path accuracy of 4.9 cm. Additionally, a VAE disentanglement model, analysed with a dedicated dataset, reports a MIG score of 0.2 and a path accuracy of 5.5 cm, underscoring its practical relevance in generative trajectory modelling.

These findings highlight the efficacy of CNN-enhanced generative AI techniques and optimization algorithms in improving precision and adaptability in robotic trajectory planning, contributing significantly to advancements in the field.